

华中科技大学

本科毕业设计[论文]

安卓应用保活行为识别与意图分析

院 系	电子信息与通信学院
专业班级	种子 2101 班
姓 名	罗畅
学 号	U202113940
指导教师	袁巍

2025 年 5 月 9 日

学位论文原创性声明

本人郑重声明：所呈交的论文是本人在导师的指导下独立进行研究所取得的
研究成果。除了文中特别加以标注引用的内容外，本论文不包括任何其他个人或
集体已经发表或撰写的成果作品。本人完全意识到本声明的法律后果由本人承担。

作者签名：年 月 日

学位论文版权使用授权书

本学位论文作者完全了解学校有关保障、使用学位论文的规定，同意学校保
留并向有关学位论文管理部门或机构送交论文的复印件和电子版，允许论文被查
阅和借阅。本人授权省级优秀学士论文评选机构将本学位论文的全部或部分内
容编入有关数据进行检索，可以采用影印、缩印或扫描等复制手段保存和汇编本学
位论文。

本学位论文属于 1、保密 ☐ ，在 年解密后适用本授权书。

2、不保密 ☐ 。

（请在以上相应方框内打“√”）

作者签名：年 月 日

导师签名：年 月 日

摘 要

随着Android智能手机的广泛应用，用户对电池续航、隐私安全及系统流畅性等问题的关注度与日俱增。为延长Android应用的存活时间，不少开发者采用了保活（Keep-Alive）技术，使其应用能长时间在后台保持活跃状态。然而，保活技术一旦被滥用，可能会导致隐私泄露、电池续航下降、发热卡顿等问题。因此，如何识别Android应用的保活行为并分析保活行为是否有不良用意就显得尤为重要。目前，学术界对Android应用保活行为的研究才刚刚兴起，缺乏有效的保活行为识别和意图分析的有效方法。为此，本文设计并实现了KALAD（Keep-ALive Application Detector）框架，对Android应用中的三类典型保活行为（无声音频、单像素Activity、灰色前台Service）进行检测与意图分析。

KALAD框架包含预处理、保活行为检测、函数调用图抽取和保活行为分析理解四个模块，其使用规则检测三种保活行为，并利用本文所提出的层级函数调用图算法抽取相关代码及其上下文，借助大模型进行多轮分析以判断保活行为意图的良恶性。实验表明，KALAD对三类保活行为检测准确率接近100%，意图良恶判断准确率最高达到88.39%。对在野的971个真实APK样本分析表明，多数保活行为与应用的功能相关，少数存在恶意可能。

综上所述，KALAD框架为保活行为的检测与意图分析提供了新思路，有助于改善用户体验，促进移动生态健康发展。

关键词：Android 保活行为；静态分析；大语言模型；意图理解

Abstract

With the widespread use of Android smartphones, users are paying more and more attention to issues such as battery life, privacy security, and system fluency. In order to extend the survival time of Android APPs, many developers have adopted the Keep-Alive methods to enable them to remain active in the background for a long time. However, once the Keep-Alive tech is abused, it may lead to privacy leakage, lower battery life, heating and freezing. Therefore, it's particularly important to identify the Keep-Alive behavior of Android APPs and analyze whether the Keep-Alive behavior has bad intentions. Currently, academic research on Android APP Keep-Alive behavior has just begun, and there is a lack of effective methods for Keep-Alive behavior identification and intent analysis. To this end, this paper presents the design and implementation of KALAD (Keep-ALive Application Detector), a framework that detects and analyzes three typical keep-alive behaviors in Android APPs (Silent Audio, Single Pixel Activity, and Gray Foreground Service).

The KALAD framework includes four modules: Preprocessing, Keep-Alive behavior detection, Call graph extraction and Behavior Analysis. It uses rules to detect three types of Keep-Alive behaviors, applies the hierarchical function call graph algorithm proposed in this paper to extract related code and its context, and uses LLM for multi-round analysis to determine the benign or suspicious intentions of them. Experiments show that KALAD achieves near-perfect detection accuracy ($\sim 100\%$) for all three keep-alive methods, and with up to 88.39% accuracy in benign/suspicious classification. Analysis of 971 real APK samples in the wild shows that most keep-alive behaviors are related to the functions of the application, and a few may be malicious.

In summary, the KALAD framework provides new ideas for detection and intent analysis of keep-alive behaviors, which helps improve user experience and promote the healthy development of the mobile ecosystem.

Key words: Android keep-Alive; Static Analysis; Function Call Graph; Intent Understanding

目 录

摘要	I
Abstract	II
1 绪论	1
1.1 研究背景与研究意义	1
1.2 国内外研究现状	2
1.3 存在的问题与挑战	4
1.4 研究内容	5
1.5 论文组织架构	5
2 相关技术及理论	7
2.1 安卓平台相关知识	7
2.1.1 Android 操作系统架构	7
2.1.2 Android 四大组件	8
2.2 安卓 APK 与逆向工程	9
2.2.1 Android APK 文件结构	10
2.2.2 Android 逆向工程相关工具	10
2.3 安卓系统保活行为	11
2.3.1 保活行为基本原理	11
2.3.2 后台播放无声音频文件保活	13
2.3.3 拉起单像素 Activity 保活	14
2.3.4 灰色前台 Service 保活	14
2.4 基于大模型的代码摘要生成	16
2.5 本章小结	17
3 基于层级函数调用图的保活识别与意图分析框架	18
3.1 引言	18
3.2 预处理	19
3.3 保活行为检测	21
3.3.1 后台播放无声音频保活检测	21

3.3.2 拉起单像素 Activity 保活检测	22
3.3.3 灰色前台 Service 保活检测	23
3.4 函数调用图抽取	24
3.5 保活行为理解与意图分析	26
3.6 本章小结	28
4 实验结果与分析	29
4.1 实验设置	29
4.1.1 实验环境	29
4.1.2 实验数据集	29
4.1.3 评价指标	30
4.2 实验结果	32
4.2.1 RQ1: KALAD 的有效性	32
4.2.2 RQ2: KALAD 的性能	33
4.2.3 RQ3: 三种保活行为在野特性	35
4.3 本章小结	37
5 总结与展望	38
5.1 本文小结	38
5.2 创新点与贡献	38
5.3 不足与展望	39
参考文献	40
附录	44
致谢	48

1 绪论

1.1 研究背景与研究意义

Android 操作系统是由美国 Google 公司设计并开发的开源操作系统,主要面向智能手机和平板等移动终端设备,也广泛应用于可穿戴设备、电视、车机等多个领域。Android 操作系统的生态非常庞大,根据 StatCounter 的统计数据,如图 1-1 所示,截至 2025 年 3 月,Android 操作系统依然 71.95%的份额高居智能手机操作系统市场的首位^①。根据另一个统计网站 APP Brain Statistics 的数据显示,截至 2025 年 4 月,全球最大的应用市场 Google Play 上已有约 154 万个 Android 应用程序^②。Android 凭借庞大的市场占有率和完善的软件生态,服务了全球 190 多个国家和地区的 39 亿多用户^③,是全球互联网发展的主流平台。然而,尽管 Android 系统取得了巨大成功,用户在实际使用过程中仍普遍面临诸如隐私泄露^[1]、电池续航不足^[2]、发热卡顿^[3]等体验痛点问题。已有研究指出,这些问题可能与 Android 系统中的“保活”行为密切相关。

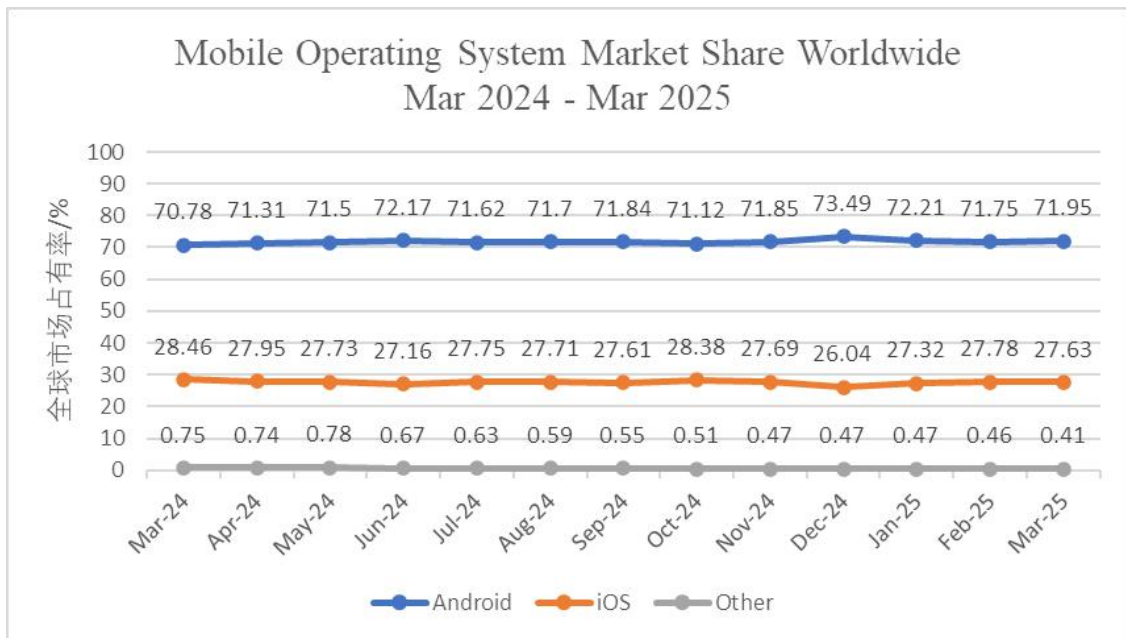


图 1-1 2024.03-2025.03 全球移动设备操作系统市场占有率统计

保活行为指的是应用程序采用各种方法来绕过 Android 系统的电池优化策略与

① 来源: <https://gs.statcounter.com/os-market-share/mobile/worldwide/#monthly-202403-202503>

② 来源: <https://www.appbrain.com/stats/number-of-android-apps>

③ 来源: <https://www.bankmycell.com/blog/how-many-android-users-are-there>

监管,使其能长期在后台保持活跃状态。为优化电池续航、减少设备发热和系统卡顿,智能手机制造商们普遍致力于限制应用的后台活动^④。然而,从 APP 的开发者的角度来看,他们往往会希望延长他们应用的活跃时间,以实现更多的功能,例如提供即时性服务(比如接受即时通讯软件的消息推送,后台持续播放音乐等),赚取利润(例如推送广告)等^[4]。广义的保活行为可以分为两类:(1) 延长 APP 在后台的存活时间,称为“保活”(Keep-Alive);(2) 使 APP 被停止后再次被唤醒,称为“拉活”(Pull-Alive)^[4]。适度、合理的良性保活行为可以给用户带来更好的体验,而过度 and 恶意的保活行为则更可能会给用户带来隐私泄露、电池加速消耗、手机发热卡顿和资费受损等问题。例如,根据腾讯安全发布的《Android 进程保护研究分析报告》^⑤,恶意开发者利用 Android 进程保活技术制造了大量流氓应用,包括流氓广告应用(占比 66%),恶意扣费软件(占比 18%)和风险软件(占比 16%)。典型案例包括 Magiclamp 病毒,该病毒伪装成破解游戏或工具软件传播,通过云端下发保活子包,强制安装应用并执行广告骚扰等恶意操作。用户一旦感染,设备会异常发热、卡顿,且难以彻底清除。

因此,在 Android 平台上对保活行为展开研究,识别保活行为模式与意图,区分良性和恶意保活行为,对提升 Android 智能手机用户体验,捍卫健康的移动应用生态发展具有重要意义。

1.2 国内外研究现状

当前国内外针对 Android 应用保活行为的研究均相对较少,但 Android 保活行为作为 Android 生态中的“灰色技术”,一直长期存在于 Android 智能设备中,并且伴随着系统版本的演进持续进化。保活技术隐蔽性显著,加之相关研究数据集较少,导致其检测与分析面临严峻挑战。当前针对 Android 保活行为的研究可分为三个主要类别:系统性检测方法的研究、特定保活技术分析和相关影响的评估,且暂无研究针对保活行为的良恶进行判断。此外,目前虽已经有多种方法能有效识别 Android 软件应用的良恶,但很少有研究针对应用的某一个特定行为进行意图分析与良恶的判定。这种研究空白使得保活技术在实际应用中存在被滥用的风险,

④ 来源: <https://dontkillmyapp.com/problem>

⑤ 来源: <https://gslab.qq.com/article-588-1.html>

需要建立更精细化的检测与评估框架。

香港理工大学的 H. Zhou 等人^[4]针对“保活”和“拉活”两种保活行为进行了系统性和开创性的研究，整理出了 6 种“杀死”应用程序进程的方法，分别提出了 7 种“保活”和 5 种“拉活”的理论方法，并用 FlowDroid 等工具构建了一个用于检测 APP 保活行为的工具——DiehardDetector（如图 1-2 所示），并用其评估了保活行为的在野普遍性。根据他们的实验结果，从 Google Play 上下载的 APP，约有 21% 使用了保活手段，证明了保活行为的在野普遍性。

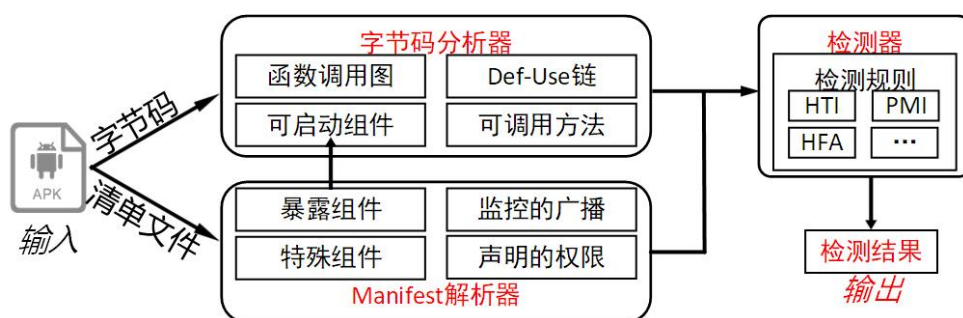


图 1-2 DiehardDetector 检测流程^[4]

中国科学技术大学的颜宏冰^[5]通过修改 Android 系统源码，结合应用对传感器的调用情况（作者认为，良性的保活软件在后台播放音频实现保活时，会结合使用传感器），对在后台播放无声音频文件以实现软件保活的行为进行了识别和检测，能有效检测应用中存在的利用无声音频文件实现恶意保活的行为。

除此之外，还有一些研究，对 Android APP 的保活行为进行了不那么系统性的研究。Y. Yan 等人^[6]在模拟器中运行应用程序，以检查应用程序在启动时是否会启动覆盖窗口以实现应用保活；Y. Shao 等人^[7]则手动研究了 23 个有保活行为的应用程序，并找到了一些保活手段，其中的大多数都旨在唤醒应用程序，即“拉活”。

也有部分研究从应用的不良行为对电池使用的影响进行了研究。如 A. Carette 等人的研究^[8]表明，APP 的不良行为会对设备的能耗即电池使用产生负面影响；Oliveira 等人^[9]和 Chowdhury 等人^[10]发现，用于开发应用程序的编程语言和运行时执行的日志会影响设备的能耗控制。

针对 Android 软件恶意行为的判别，当前的研究几乎都是研究如何判别整个 APP 的良恶性的。如 C. Gao 等人^[11]提出了一种抗混淆的安卓恶意软件检测方法，实现了高抗混淆性能；Jaehyeong Lee 等人^[12]提出了一种结合遗传算法（Genetic Algorithm, GA）进行特征选择的 Android 恶意软件的检测方法，提升了检测准确

率并减少模型构建时间，可有效检测 Android 手机中的多种恶意软件；Merve Cigdem Ipek 等人^[13]实现了一种利用图神经网络（Graph Neural Networks, GNNs）和图注意力机制（Graph Attention Mechanism）对 Android 恶意软件进行检测的方法，提升了恶意软件分析的可扩展性、可解释性和可靠性。然而，他们都没有提到对应用的某一个特定行为的意图进行良恶的判定。

纵观国内外，目前没有一项系统性的研究对 APP 保活行为和意图及其良恶区分进行深入理解与分析，故本课题针对 APP 的特定行为——保活行为进行意图理解与分析，具有开创性和重要意义。

1.3 存在的问题与挑战

通过对国内外已有研究内容的概述与对 Android 真实保活行为的调研，可以看出，学术界对 Android 保活行为的研究相比于工业界对 Android 保活技术的开发，存在一定的差距。本文对现有研究存在的问题与挑战进行如下概述：

(1) 在真实工业界中，保活技术多种多样，难以使用通用的手段检测特定的保活行为，需要编写针对性的规则进行检测。现有针对一些较为泛化保活手段的检测方案（如 H. Zhou 等人^[4]设计的 Diehard Detector），但在真实的工业场景中使用的保活手段，往往是这些较为泛化的保活手段的变体或改进版，现有的保活检测方法无法针对工业界中特定的保活手段进行检测。

(2) 在真实工业界中，APP 的功能繁杂多样，一个普通 APP 往往会集成多种多样的功能（如即时通信、定位服务、音视频处理、支付接口、推送通知、广告投放等），由此也会引起 APP 代码规模的急剧膨胀^[14]。在这样的背景下，即使成功检测到了保活行为，想要从庞大的 APP 代码中抽取出与保活行为相关的代码及其上下文，也存在较大的困难。

(3) 使用传统方法难以理解代码本身的意图与良恶。现有的恶意软件检测手段，往往比较依赖 APP 的一些特定特征，例如 APP 申请的权限^[15]、API 接口调用^[16]、组件声明^[17]、字符串特征^[18]（包括可疑 URL、加密字符串等）等，而很少会针对一段代码片段本身进行研究。保活行为只是 APP 众多行为中的一小部分，即使成功抽取到了保活行为相关代码及其上下文，也难以利用传统方法对相关代码本身进行良恶判断。

1.4 研究内容

为了应对上述困难与挑战,本文构建了一个名为 KALAD (Keep-ALive Application Detector) 的基于层级函数调用图的保活识别与意图分析框架,旨在对工业界较为流行的三种保活手段进行检测,并借助大语言模型对保活行为意图进行分析与理解。

(1) 基于静态分析的保活行为检测

在与华为安全部门的合作中,经过调研发现,在华为应用市场的 APP 中,存在最常见的三种保活行为手段: 1) 后台播放无声音频文件保活; 2) 拉起单像素 Activity 保活; 3) 灰色前台 Service 保活。本文将基于 Android 静态分析和逆向工具,构建特定规则,对如上三种保活行为分别进行检测,并以此为出发点,抽取保活行为相关代码。

(2) 基于层级的函数调用图抽取

为便于分析与理解,本文首先生成 APP 的函数调用图,该调用图可以描述 APP 内所有函数的调用与被调用关系。然而,保活行为相关的代码,并不能直接在函数调用图中展现出来。本文设计了一种基于层级的函数调用图抽取算法,以函数所在类的类名为层级,对保活行为相关的代码进行按层级精确抽取。该算法抽取到的相关代码,能较好的反应 APP 使用保活手段的意图与目的。

(3) 基于大模型的保活行为意图识别与分析

随着 AI 技术的发展,使用大语言模型生成代码摘要已经是较为成熟的技术^[19],相比于传统的基于规则或深度学习模型的代码理解方法,大语言模型具备更强的上下文理解和跨模表达能力,能够以更接近人类的理解方式对代码进行语义提取和总结。因此,本文借助大语言模型,通过设计多轮提问的方式,让大语言模型参与保活行为相关代码的理解分析,生成总结,并对保活行为的良恶性质进行辅助判断。

1.5 论文组织架构

本文共分为五个章节,具体的组织架构如下:

第一章,绪论。本章主要介绍了 Android 应用保活行为识别与意图分析的背景与研究意义,综述国内外与 Android 软件保活相关研究的现状,阐明了当前存在的

问题与挑战，简述了本文的主要研究内容与整体结构。

第二章，相关技术与理论。本章首先介绍了 Android 操作系统的相关知识，包括 Android 操作系统架构和 Android 四大组件，接着介绍了 Android APK 结构和 Android 逆向工程相关知识与工具，然后介绍了 Android 保活行为相关知识与工业界三种常用保活行为的实现方式与原理，最后介绍了大模型生成代码摘要的相关知识。这些理论与技术，为后续的框架设计搭建提供了理论基础与技术支撑。

第三章，基于层级函数调用图的保活识别与意图分析框架。本章首先总览介绍了 KALAD 框架，包括预处理、保活行为检测、函数调用图抽取、保活行为理解分析四个部分，接着依次详细介绍了各个部分的流程与实现。本文通过搭建 KALAD 框架，可系统化、自动化地分析 APP 的三种特定保活行为，并利用大模型对保活行为进行理解、分析与良恶判别。

第四章，实验结果与分析。本章围绕 KALAD 的有效性与性能展开了系统性的评估实验，对 KALAD 的有效性、性能及影响因素以及三种保活行为的在野特征进行了详细评估。实验结果表明，KALAD 对三类保活行为检测准确率接近 100%，良恶判断准确率达到 88.39%，APK 包含的保活行为数量会显著影响 KALAD 的分析时间，使用 Androguard 进行预处理阶段的耗时则主要与 APK 包含的函数数量有关。对在野的 971 个真实 APK 样本分析则表明，多数保活行为与应用的功能相关，少数则存在泄露用户隐私等恶意可能。

第五章，总结与展望。本章系统性的总结了研究工作的创新贡献和局限性。主要创新与贡献包括 1) 构建了一个基于层级函数调用图的保活行为检测与分析理解框架；2) 提出了一种以类名 (ClassName) 为层级的函数调用图抽取算法。同时，本章也客观指出了本文仍存在无法分析加壳 APK、检测范围仅覆盖 3 种保活行为以及大语言模型理解精度不足等方面的不足，为后续研究指明了方向。

2 相关技术及理论

2.1 安卓平台相关知识

本节将系统性介绍 Android 平台的核心架构与“四大组件”机制，这是理解保活行为技术实现的基础前提。首先介绍 Android 系统的分层结构设计，依次阐述其从 Linux 核心层到上层应用的原理和功能；其次详解 Android 四大组件的功能特性与交互机制。这些基础知识将为后续进行保活行为识别提供必要的理论支持。

2.1.1 Android 操作系统架构

Android 系统是 Google 公司设计开发的操作系统，由 Linux 系统发展而来^[20]。如图 2-1 所示，Android 系统采用分层结构^[21]，由 Linux 内核、硬件抽象层、Native C/C++库、Android Runtime、Java API 框架和应用程序这几个关键层次组成^[22]。

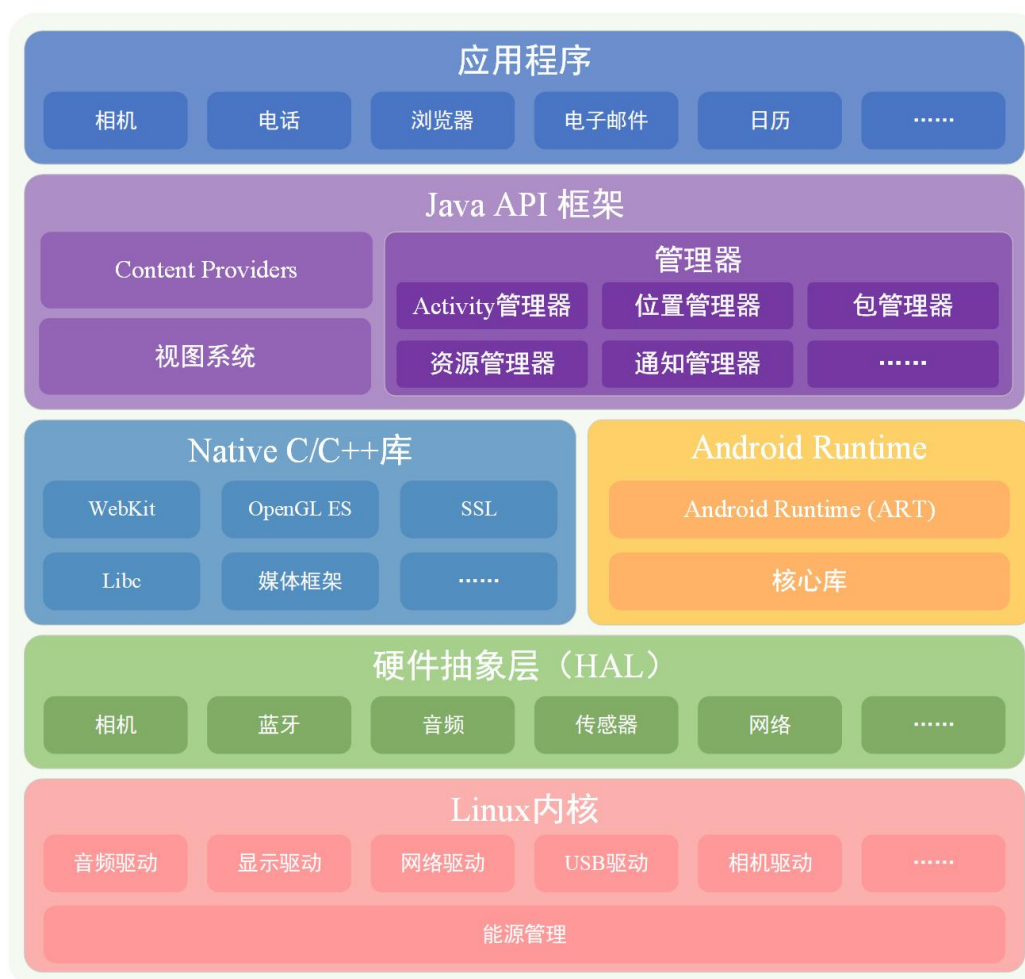


图 2-1 Android 系统架构图

Linux 内核是安卓系统的核心部分,是 Android 与所有硬件进行交互的基础^[23]。Linux 内核负责底层硬件设备的管理与驱动,包括音频、显示、网络等,同时, Linux 内核也会承担进程管理、内存管理、能源管理等关键任务。

硬件抽象层(HAL)则提供了用于向较高级别的 Java API 框架公开设备硬件功能的标准接口^[24]。HAL 包含多个库模块,如相机、蓝牙、传感器等。每个模块都为特定的硬件组件实现一个接口,上层模块可通过对应接口访问硬件组件的库模块。

系统运行库由 Native C/C++库和 Android Runtime 组成。其中 Native C/C++库由 C/C++编写,包括标准 C 库(如 libc)、用于网页浏览的 Webkit、实现图像渲染的 OpenGL ES 等。这些库可向上层提供高性能、低延迟和直接的硬件访问能力,对处理高性能任务至关重要^[25]。Android Runtime 则包括虚拟机和核心库两个部分,采用了预编译(AOT)和即时编译(JIT)相结合的方式^[26],主要供应用与框架层调用使用。

Java API 框架层则为应用开发者提供了一套可复用的 API,涵盖 Content Provider、支持 UI 开发的视图系统、Activity 管理等模块^[27],开发者通过调用这些 API 便捷的管理系统资源,管理 Android 组件生命周期并实现 APP 的各项功能交互^[28]。

最上面一层则是应用程序层,即为用户可以操作的 APP,例如电话、浏览器、日历、相机等。应用程序层通过调用 Java API 框架提供的接口,与底层系统组件进行交互,从而实现各种功能。

2.1.2 Android 四大组件

Android 应用程序由若干功能模块组成,其中最核心的模块即为被称为 Android “四大组件”^⑥的四类基础构建单元(如图 2-2 所示),分别是 Activity(活动)、Service(服务)、BroadcastReceiver(广播接收器)和 ContentProvider(内容提供者),这四类组件共同构成了 Android 应用程序的功能架构和交互逻辑,下面将依次介绍各个组件的功能与特性。

(1) Activity。Activity 是用于构建用户界面的核心组件,每一个 Activity 通常

⑥ 来源: <https://developer.android.com/guide/components/fundamentals?hl=zh-cn>

都对应一个用户可见的界面，Activity 负责处理用户的交互行为，例如响应点击，呈现数据等。

(2) Service。Service 是一种在后台运行的组件，适合执行长期运行任务（例如，后台播放音乐，导航定位等）或网络处理任务等。与 Activity 不同的是，Service 不提供界面，不直接与用户交互。

(3) BroadcastReceiver。BroadcastReceiver 用于响应来自系统（例如，息屏广播、电池电量变化广播）或其他应用程序的广播事件，是一种事件驱动型的组件，其可以在不启动应用界面的情况下，对特定事件做出快速响应。

(4) Content Provider。Content Provider 是一种用于实现不同应用之间数据贡献的组件，通过统一的 URI 接口，其他应用可以通过 Content Provider 对其提供的数据（例如，系统应用“电话”提供联系人信息）进行访问。

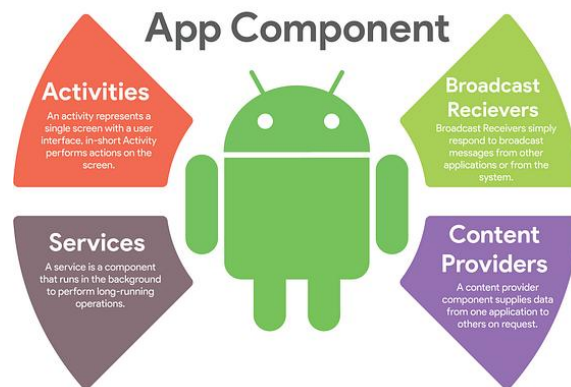


图 2-2 Android 四大组件^⑦

在 Android 可以启动组件之前，应用开发者必须在 APP 的清单文件 AndroidManifest.xml 中声明对应组件^[29]，以便系统可以通过读取其清单文件确定组件是否存在。

Android 应用程序可以使用 Java API 框架中提供的名为 Intent 的异步消息 API，启动四种组件中的其中三种 (Activity, Service, BroadcastReceiver)，Content Provider 则会在被来自 ContentResolver 的请求定位时激活^[30]。

2.2 安卓 APK 与逆向工程

本节将重点解析 Android 应用安装包的文件结构和逆向工程的相关知识，这是

^⑦ 来源: <https://medium.com/@Abderraouf/understand-android-basics-part-1-application-activity-and-lifecycle-b559bb1e40e>

实现保活行为静态分析检测的技术基础。首先系统性解析 APK 文件的组织结构与关键组成部分，特别关注 Manifest 清单文件和字节码等核心要素；其次介绍逆向工程的工具选型依据和功能特性，并说明他们在保活行为分析的具体应用场景。这些技术将为后续构建自动化检测框架提供必要的技术支持和理论指导。

2.2.1 Android APK 文件结构

APK (Android PacKage^⑧) 是 Android 应用软件的安装包文件格式，用于分发和安装 Android 移动应用及中间件，其包含了安装与运行 Android 应用程序所需的必要文件与资源^[31]。表 2-1 展示了 APK 文件中包含的关键文件/文件夹及其描述。

表 2-1 Android APK 文件结构及其描述

文件/文件夹	描述
AndroidManifest.xml	清单文件，记录关于应用的必要信息,包括应用包名等
smali/	用于存放字节码，可能存在多个，使用 smali 格式表示
res/	APP 所需的资源文件夹，包含应用图标、界面布局等信息
assets/	用于存放未经过编译的原始文件
META-INF/	用于存放 APK 的数字签名和证书等相关的文件
lib/	用于存放 native 库文件
resources.arsc	资源映射文件，包含 res/目录中所有资源的索引和相关信息

在 APK 包含的文件结构中，最重要的就是清单文件 AndroidManifest.xml 和 smali/文件夹中包含的字节码文件。目前主流的 Andoid 恶意软件检测算法，就是主要从这两个部分提取静态特征并进行分析检测^[32]，而本文所搭建的保活检测识别与意图分析框架，也将从这两方面入手。

2.2.2 Android 逆向工程相关工具

逆向工程是一种分析目标系统的过程，其目的是识别出系统的各个组件以及它们之间的关系，并以其它的形式或在较高的抽象层次上，重建系统的表征^[33]。对于 Android 来说，逆向工程则是对已发布的 Android 应用程序的 APK 文件进行分析与破解，以了解 APP 内部工作原理。

本文使用的 Android 逆向工程工具有如下几种：

(1) APKTool^⑨，一个用于解包 APK 文件，提取资源文件和 smali 代码以及重打

⑧ 来源：<https://zh.wikipedia.org/wiki/APK>

⑨ 来源：<https://apktool.org/>

包 APK 文件的轻量化工具；

(2) Androguard^⑩，一个用于处理 Android APK 的完整工具，其核心功能包括 APK 解析与反编译、DEX 文件分析、函数调用图生成等，除此之外，Androguard 也支持作为一个 Python 库，使用 Python 代码进行调用，从而支持批量化 APK 检查任务；

(3) Yara^⑪是一款旨在（但不限于）帮助恶意软件研究人员识别和分类恶意软件样本的工具，可使用 Yara 基于文本或二进制创建任意字符串特征匹配的描述规则，从而高效快速的匹配得到想要检查的 APP 特征。

本文将借助以上工具构建保活识别与意图分析框架 KALAD 的第一部分——保活行为检测。

2.3 安卓系统保活行为

Android 保活行为指的是 Android 应用开发者，通过编写具有特定逻辑的程序代码，绕过 Android 操作系统的监管，从而人为延长其应用程序在后台的存活时间。本章将首先从 Android 操作系统与进程管理层面介绍 Android 保活行为，并从代码角度分别介绍目前工业界中应用较为广泛的三种保活手段及其实现方式。

2.3.1 保活行为基本原理

Android 操作系统基于 Linux 发展而来，其资源调度操作的基本单位是进程（Process）^[34]，Android 应用绕过操作系统的监管，本质上是绕过操作系统对其持有的进程的监管。一个 Android 应用程序可能会有一个或多个进程，在默认情况下，Android 应用程序的组件在 APP 的主进程中运行，但应用开发者可以在清单文件中指定其进程属性，使其可以在主进程以外的进程中运行。如果 Android 应用程序的所有运行进程都被清理掉，则该应用程序视为停止，反之则视为存活。

在 Android 系统中，存在一个与进程管理相关的名为 ProcessRecord^⑫的数据结构，Android 系统为每一个进程创建 ProcessRecord 实例，并用其表征与管理应用

⑩ 来源：<https://github.com/androguard/androguard>

⑪ 来源：<https://virustotal.github.io/yara/>

⑫ 来源：<https://android.googlesource.com/platform/frameworks/base/+/master/services/core/java/com/android/server/am/ProcessRecord.java>

程序的进程。Android 操作系统对进程的管理，主要依赖 ProcessRecord 中存在的两个与进程调度优先级相关的字段：curadj 和 curschedgroup，分别用于表征进程的 OOM 调整值（Out-Of Memory Adjustment Value）与进程调度组值（Thread Scheduling Group）。具体而言，具有较高优先级的应用程序进程通常具有较小的 OOM 调整值，使其被 Android 清理的可能性变小；具有相同优先级的进程往往会被安排到同一个线程调度组中监管。

表 2-2 总结了不同应用程序进程的优先级可能情况，总结来说，持有前台组件、可件组件或可感知组件的应用程序进程，往往具有较小的 OOM 调整值，和更高的线程调度组，从而获得更高的优先级，以减小甚至消除被 Android 操作系统资源调度管理机制清理的风险。

表 2-2 Android 系统中应用程序进程优先级值可能情况总结^[4]

OOM 调整值	值	举例	线程调度组
前台应用	0	P0: 进程持有前台 Activity，且设备处于唤醒状态	顶层应用(3)
		P1: 进程持有前台 Activity，但设备处于休眠状态	后台(0)
		P2: 进程持被前台应用绑定的 Service	默认(2)
		P3: 进程持有被前台应用需要的 Content Provider	默认(2)
可见应用	100	P4: 进程持有可见 Activity	默认(2)
	—	P5: 进程持有被可见应用绑定的 Service	默认(2)
	200	P6: 进程持有被可见应用需要的 Content Provider	默认(2)
可感知应用	200	P7: 进程持有前台 Service	默认(2)
		P8: 进程持有悬浮窗口	默认(2)
		P9: 进程持有被可感知应用绑定的 Service	默认(2)
		P10: 进程持有被可感知应用需要的 Content Provider	默认(2)
服务	500	P11: 进程持有有一个 30 分钟内启动的 Service	后台(0)
缓存应用	900	P12: 进程持有有一个被停止的 Activity	后台(0)

在工业界的实际应用中，有许多保活手段实现方法，但这些方法几乎都是表 2-2 中方法的具体实现。在与华为安全部门的合作中，经过调研发现，在华为应用市场的 APP 中，存在的最常见的三种保活行为手段：1) 后台播放无声音频文件保活，该手段属于 P9 的实际应用，如果结合前台 Service 的使用，则也属于 P7 的实际应用；2) 拉起单像素 Activity 保活，该手段属于 P4 的实际应用；3) 灰色前台 Service 保活，该手段则是利用系统漏洞，属于 P7 的实际使用。基于华为安全部门的调研数据，本文将聚焦这三种保活行为作为研究对象，构建能够识别并分析理解这三种保活行为的框架。后续 2.3.2~2.3.4 节将详细介绍这三种保活行为的实现原理，为检测框架的构建奠定理论基础。

2.3.2 后台播放无声音频文件保活

在一般情况下, Android 系统为了保障用户体验, 会将播放音频视为用户可感知的活动, 所以会尽量避免打断正在播放音频的应用^[35] (如音乐、语音、视频等), 即使这些应用切换到了后台。系统会给播放音频的应用进程较高的优先级, 避免其被杀死。所以, 在后台播放无声的音频文件可以欺骗 Android 系统, 让系统判断应用正在播放媒体, 但用户不会听到声音, 以此实现后台常驻且不会引起用户反感或警觉。除此之外, 无声音频播放的行为往往会结合前台 Service 一起使用, 使得应用进程更加难以被系统发现和清理。

图 2-3 展示了后台播放无声音频文件实现保活的简化代码示例。在该 Service 组件中, 开发者使关键类 MusicService 继承 android.app.Service 类, 并重写 onStartCommand()方法, 在其中调用 MediaPlayer.create()方法, 加载一段预置于应用内的无声音频资源文件 R.raw.silent_audio, 并用方法 setLooping(true)设置其为循环播放; 除此之外, 开发者还将 onStartCommand()方法返回值设置为 START_STICKY, 这将使得 Service 被系统终止后, 自动尝试重启。这些操作将使系统认为应用正在执行音频播放任务, 即使该音频是无声的, 也可让系统将进程调度视为“可感知应用”级别, 从而显著降低其被系统清理的可能性。

```
1 public class MusicService extends Service {
2     private MediaPlayer mMediaPlayer;
3
4     @Override
5     public int onStartCommand(Intent intent, int flags, int startId) {
6         mMediaPlayer = MediaPlayer.create(this, R.raw.silent_audio);
7         mMediaPlayer.setLooping(true);
8         mMediaPlayer.start();
9         return START_STICKY;
10    }
11
12    @Override
13    public void onDestroy() {
14        if (mMediaPlayer != null) {
15            mMediaPlayer.stop();
16            mMediaPlayer.release();
17        }
18        super.onDestroy();
19    }
20 }
```

图 2-3 无声音频保活实现简化代码实例

2.3.3 拉起单像素 Activity 保活

由表 2-2 可知，系统一般会给前台应用和可见应用设置较高的优先级，从而降低被清理的概率。为降低被操作系统清理的概率，开发者可通过在本进程开启一个可见 Activity 以提升进程优先级。而为了不引起用户反感与警觉，开发者将该 Activity 的大小设置为一个像素，并透明且无切换动画。

```

single_pixel
1  // OnePixelActivity.java
2  public class OnePixelActivity extends Activity {
3      @Override
4      protected void onCreate(Bundle savedInstanceState) {
5          super.onCreate(savedInstanceState);
6          Window window = getWindow();
7          WindowManager.LayoutParams params = window.getAttributes();
8          params.x = 0;
9          params.y = 0;
10         params.height = 1;
11         params.width = 1;
12         window.setAttributes(params);
13     }
14 }
15
16 // Somewhere else to start the OnePixelActivity
17 public void startOnePixelActivity(Context context) {
18     Intent intent = new Intent(context, OnePixelActivity.class);
19     intent.addFlags(Intent.FLAG_ACTIVITY_NEW_TASK);
20     context.startActivity(intent);
21 }
    
```

图 2-4 拉起单像素 Activity 保活实现简化代码示例

图 2-4 展示了拉起单像素 Activity 实现保活的简化代码示例。在该 Activity 组件中，开发者使关键类 OnePixelActivity 继承 android.app.Activity 类，并重写 onCreate() 方法，在其中通过 setAttribute 方法，将该 Activity 的位置设置为屏幕左上角 (x=0;y=0)，宽高均设置为 1 (height=1;width=1)，从而该 Activity 设置为一个隐藏在屏幕左上角的单像素 Activity。该 Activity 对用户几乎不可见，但将被 Android 系统视为前台可见状态，从而将进程调度视为“可见应用”级别，从而显著降低其被系统清理的可能性。

2.3.4 灰色前台 Service 保活

前台 Service 是 Android 中一种具有较高系统优先级的 Service 类型，其特点是即使应用处于后台，也可以较为稳定地运行，不容易被系统清理。前台 Service 常见于需要连续执行的任务，例如音乐播放、导航定位、运动记录、大文件下载等。

如图 2-5 所示, 前台 Service 的启动通常伴随着通知栏的通知^⑬, 以明确告诉用户后台正在运行的任务。

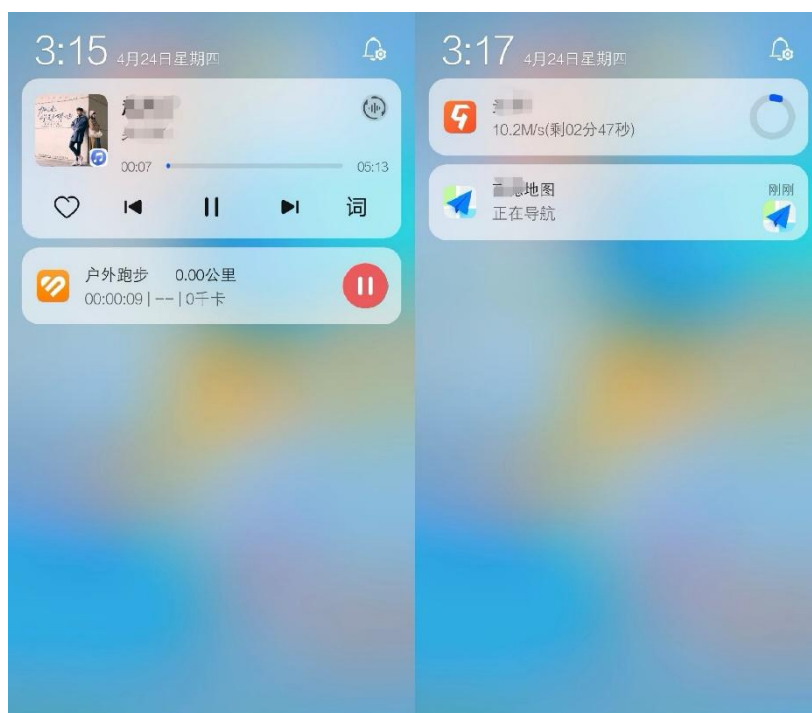


图 2-5 Android 前台 Service 示例

由表 2-2 可知, 持有前台 Service 的进程可以提高自身优先级, 而启动前台 Service, 则必须调用 `android.app.Service` 中的 `startForeground(int id, Notification notification)` 方法以在通知栏显示通知图标。对于 Android API < 18 的设备, Android 允许通过直接向方法 `startForeground()` 的第二个参数传入一个空 `Notification` 对象以启动前台 Service, 该进程虽然被置于前台, 但由于传入的 `Notification` 对象为空, 通知栏并不会展示图标, 系统也不会对该进程进行清理。而对于 Android API ≥ 18 (Android 4.3+) 的设备, Android 限制前台服务必须显示通知栏图标, 即 `startForeground()` 函数的第二个参数必须传入有效的 `Notification` 实例, 但开发者可以通过如下的灰色逻辑绕过该限制:

1. 先启动 ServiceA (主 Service), 调用函数 `startForeground(ID, Notification)`;
2. 立刻再次启动 ServiceB (辅 Service), 也调用 `startForeground(ID, Notification)`, 且通知 ID 与上一步中的通知 ID 保持一致;
3. 然后立即在 ServiceB 中调用 `stopSelf()` 或 `stopService()` 函数以停止 ServiceB;
4. 此时, 虽然 ServiceA 仍在运行, 但是系统却因为通知 ID 被回收而移除了

^⑬ 来源: <https://developer.android.com/develop/background-work/services/fgs>

通知图标。

图 2-6 展示了使用上述逻辑实现灰色前台 Service 保活的简化代码，可将持有 ServiceA 的进程优先级提高至“可感知应用”级别，但因为移除了通知栏图标，该 Service 对用户不可见，从而显著降低其被系统或用户清理的可能性。由于为 Android API<18（Android4.2 及以下）设备开发的 APK 现在已经几乎不存在了，故本文只考虑 Android API>18（Android4.3+）的情况。



```

1 // ServiceA.java
2 public class ServiceA extends Service {
3     private static final int NOTIFICATION_ID = 1001;
4     @Override
5     public void onCreate() {
6         super.onCreate();
7         noti = createNotification(); //模拟创建notification
8         startForeground(NOTIFICATION_ID, noti);
9         // 启动ServiceB
10        startService(new Intent(this, InnerService.class));
11    }
12    @Override
13    public IBinder onBind(Intent intent) {
14        return null;
15    }
16 }
    
```

```

1 // ServiceB.java
2 public class ServiceB extends Service {
3     private static final int NOTIFICATION_ID = 1001;
4     @Override
5     public void onCreate() {
6         super.onCreate();
7         noti = createNotification(); //模拟创建notification
8         startForeground(NOTIFICATION_ID, noti);
9         // 立即停止自身
10        stopSelf();
11    }
12    @Override
13    public IBinder onBind(Intent intent) {
14        return null;
15    }
16 }
    
```

图 2-6 灰色前台 Service 保活实现简化代码示例

2.4 基于大模型的代码摘要生成

代码摘要（Code Summarization）是一种典型的代码理解任务，旨在自动生成简介准确的自然语言描述，帮助人们快速理解代码的功能和意图。传统的代码摘要方法主要依赖规则、模版或基于编码器-解码器结构的神经网络模型，虽然在一定程度上提升了自动化程度，但在摘要的语义一致性和上下文理解等方面仍然存在一定局限性。

近年来快速发展的大语言模型（Large Language Model，简称 LLM、大模型，下同）如 ChatGPT、Deepseek 等在自然语言处理和代码生成等领域也有了突破性进展。得益于大规模语料训练与深层神经网络结构，这些模型展现出强大的代码理解能力。相较于传统方法，大模型不仅能够处理更复杂的代码结构，还具备更强的跨语言迁移和少样本学习能力，使其在代码摘要任务中表现出色。除此之外，基于大模型微调的各类模型如 CodeX^[36]、CodeT5+^[37]、CodeBERT^[38]等，在代码摘要生成工作上也有了非常良好的表现。

本文不研究如何基于大模型进行微调，而是直接使用大模型的代码摘要生成能力，帮助分析和理解保活行为的相关代码。

2.5 本章小结

本章对本文涉及的相关技术及理论进行了详细介绍。首先,本章从 Android 操作系统架构和 Android 应用的“四大组件”入手,介绍了 Android 平台的相关基础知识,包括 Android 系统的分层结构和四大组件的详细功能,这是理解 Android 软件应用的必要基础。然后,本章从 Android APK 结构和 Android 逆向工具两个方面介绍了 Android 逆向工程的必要知识,这是后续对 APK 文件进行静态分析以检测保活行为的坚实基础。接着,本章从进程视角介绍了 Android 保活行为的原理知识,并分别详细介绍了三种保活方式的原理与代码实现,为本文后续介绍的三种保活行为检测方法提供理论依据。最后,本章还介绍了大模型代码摘要生成的相关知识,这将是本文分析与理解保活行为相关代码的有力武器。

本章内容为后续第三章提出的基于层级函数调用图的保活行为检测理解框架的设计与实验提供了重要的理论依据与技术支持。

3 基于层级函数调用图的保活识别与意图分析框架

3.1 引言

为准确检测 Android APK 中的保活行为，并对这些行为进行深入分析与理解，本文设计并实现了一个名为 KALAD（Keep-ALive Application Detector）的基于层级函数调用图的保活识别与意图分析框架，该框架综合应用了静态分析、规则检测与大模型理解等技术手段，可以系统化、自动化地完成特定保活行为的检测与意图分析。KALAD 的总体架构设计如图 3-1 所示，主要由四个模块组成。

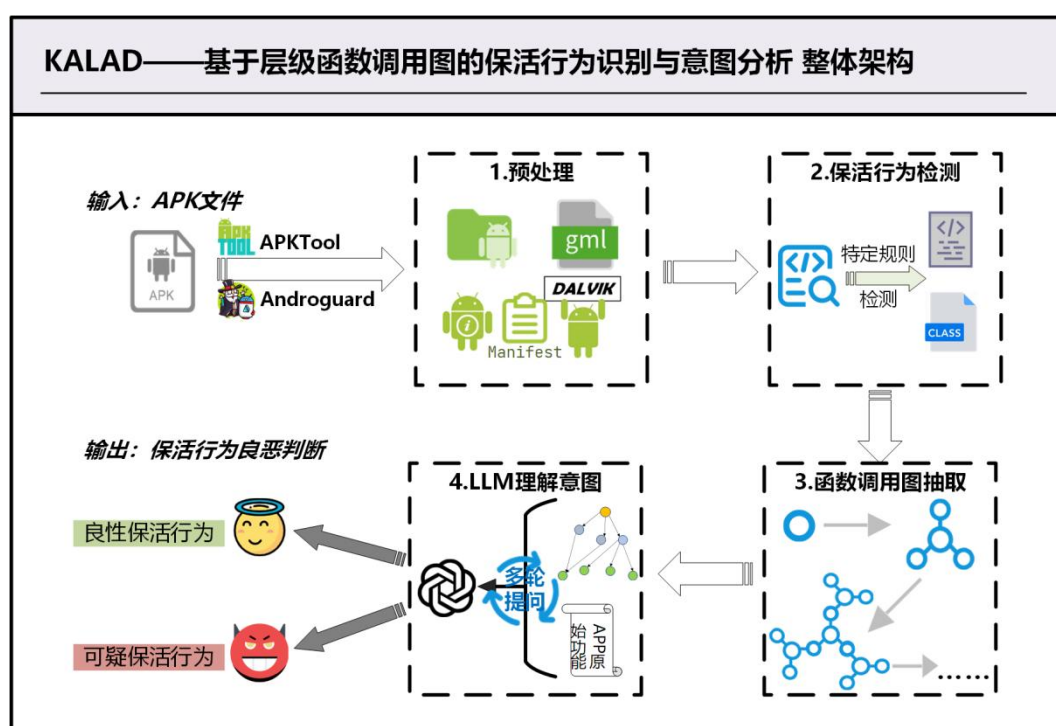


图 3-1 基于层级函数调用图的保活识别与意图分析框架整体架构图

(1) 预处理。对输入的 APK 进行预处理，包括使用 APKTool 对 APK 文件进行反编译（解包），使用 Androguard 生成 APP 的完整函数调用图，提取 APP 的相关信息（AndroidManifest.xml、APP 包名、版本号、组件声明等），解析 APK 的字节码文件等。

(2) 保活行为检测。针对工业界中常见的三种保活行为：1) 无声音频保活、2) 单像素 Activity 保活、3) 灰色前台 Service 保活，分别编写保活行为检测规则，通过匹配资源 ID、关键 Java API 框架调用、组件设置等信息，对保活行为进行检测，得到与上述保活行为相关的关键函数与关键类，为进一步抽取函数调用图和理解

分析打下基础。

(3) 函数调用图抽取。以类名 (ClassName) 为层级，基于设计的函数调用图抽取算法，从上一步中找到的保活行为关键函数与关键类出发，从 APP 的总函数调用图中抽取保活行为相关函数代码节点、类及上下文，最终得到与保活行为相关的函数调用子图。该子图不仅保留了关键保活行为代码逻辑，还完整展现了相关调用链，有助于后续基于大模型的代码理解过程。

(4) 使用 LLM 理解保活意图。通过多轮提问的方式，逐步向大模型输入抽取得到的函数代码及上下文信息，每一轮提问聚焦于不同层次的理解目标，让其逐步理解函数代码，得到代码摘要，进一步对保活行为意图进行理解与总结，并最终对保活行为良恶性进行判断。

3.2 预处理

为便于后续检测与处理，需要对 APK 进行预处理，本小节将介绍在对 APK 文件进行分析之前，需要做的预处理工作，包括如下几步：

(1) 对 APK 进行解包。使用 APKTool 对 APK 进行解包，得到 APK 的解压文件夹，由表 2-1 可知，APK 解包文件夹中包括使用 smali 格式表示的 APK 字节码文件，这些字节码文件对后续的代码分析与理解十分重要。

(2) 生成 APK 的函数调用图。使用 Androguard 生成 APK 的函数调用图，生成的函数调用图文件为.gml 格式（如图 3-2 所示）。

```

1  graph [
2    directed 1
3    node [
4      id 0
5      label "La/a;->addOnContextAvailableListener(La/c;)V [access_flags=public abstract] @ 0x0"
6      external 0
7      entrypoint 0
8      methodname "addOnContextAvailableListener"
9      descriptor "(La/c;)V"
10     accessflags "public abstract"
11     classname "La/a;"
12   ]
13 > node [...]
22 > node [...]
32 > node [...]
33   .....
34   edge [
35     source 294416
36     target 4082
37   ]
38 > edge [...]
41 > edge [...]
42   .....
43 ]

```

图 3-2 Androguard 生成的函数调用图文件示例

完整的函数调用图 GML 文件包括三个部分：

1. `directed` 标注, 1 代表该图为有向图, 0 代表无向图;
2. 多个 `node`, 其中每一个 `node` 都代表一个函数, `node` 结构中包括多个字段和描述:
 - `id`: 该函数节点在该图中的唯一标识符;
 - `label`: 该函数节点的标签, 包括函数所在类、函数名称、函数签名、访问权限标记等信息;
 - `external`: 表示是否为外部方法 (例如 Java API 框架中的代码), 0 表示非外部方法, 1 反之;
 - `entrypoint`: 表示是否为入口点 (如四大组件中的 `onCreate`、`onStart` 等方法), 0 表示不是入口点, 1 反之;
 - `methodname`: 函数的简单名称;
 - `descriptor`: 函数签名, 包含参数与返回值;
 - `accessflags`: 函数访问权限, 如 `public`、`private`、`abstract` 等;
 - `classname`: 函数所在类的名称。
3. 多条边, 其中每一条边都代表两个函数之间的调用关系, `edge` 结构中包括 `source` 和 `target` 两个字段, `source` 字段为调用方函数节点的 `id`, `target` 字段为被调用方函数节点的 `id`。

(3) 获取 APK 的 Dalvik 可执行文件 (Dalvik Executable, 简称 DEX) 列表和 APK 相关信息。为便于后续对 APK 的字节码进行分析, 可以调用 Androguard 的函数 `AnalysisAPK()` 对 APK 文件进行预处理。`AnalysisAPK()` 是 Androguard 中的一个高级封装函数, 其作用是使用一行命令完成对 APK 文件的解包、解析和静态分析, 并返回多个重要的中间对象。`AnalysisAPK()` 函数返回值为一个三元组 (`apk,dvm,analysis`), 其中 `apk` 是类 `androguard.core.apk.APK` 的实例, 其解析了 APK 的基本信息, 包括 Manifest 文件、包名、版本、权限等; `dvm` 是一个列表, 其解析了 APK 中包含的 DEX 字节码 (作为一个列表), 便于后续进行静态代码分析。`Analysis` 则是代码的静态分析结果, 包括代码交叉引用、变量分析等。

通过上述步骤, 对 APK 文件进行了较为全面的预处理, 不仅提高了 KALAD 的处理速度, 更为后续分析与处理步骤提供良好基础。

3.3 保活行为检测

本节将依次介绍针对 2.3 节中展示的三种常见保活行为,编写的对应检测规则,使用这些规则,可以准确、高效、自动化地检测出一个 APK 中存在的对应保活行为,并找出其保活行为的关键类与函数。

3.3.1 后台播放无声音频保活检测

图 3-3 展示了检测后台无声音频保活行为的基本流程。KALAD 的无声音频保活行为检测流程分为三步:



图 3-3 检测无声音频保活行为基本流程

(1) 寻找无声音频文件。

后台播放无声音频以实现保活,其核心是无声音频文件,而声波的强度与音频振幅的平方成正比^[39],故首先需要在 APK 解包文件夹中寻找音频文件,并检查其平均振幅是否在设定的阈值之下,如果是,则判定其为无声音频文件。KALAD 使用式 3-1 计算音频文件的平均振幅:

$$\bar{A} = \frac{1}{N} \sum_{n=0}^{N-1} \left| \frac{x[n]}{2^{S-1}} \right| = \frac{1}{N \times 2^{S-1}} \sum_{n=0}^{N-1} |x[n]| \quad (3-1)$$

其中 N 为音频文件的总采样点, S 为音频文件采样位深, $x[n]$ 为音频文件的第 n 个采样点 ($n=0,1,2,\dots,N-1$)。

KALAD 将平均振幅小于阈值 0.003 的音频文件视为无声音频文件,并针对该文件进行进一步检测:

(2) 匹配无声音频文件对应全局资源 ID。

由 2.2.1 节可知,APK 文件中的 `res/` 文件夹是 APP 所需的资源文件夹,包含应用图标、界面布局等信息,在 `res/value/` 文件夹中,存在一个名为 `public.xml` 的 XML 文件(如图 3-4 所示),记录了 APP 所需的所有资源文件的全局资源 ID。在该文件中匹配找到的无声音频文件,并记录其全局资源 ID。

```

1  <?xml version="1.0" encoding="utf-8"?>
2  <resources>
3      <public type="anim" name="ttlive_slide_out_top" id="0x7f01009e" />
4      .....
5      <public type="attr" name="yearTodayStyle" id="0x7f0405ee" />
6      .....
7      <public type="color" name="viewfinder_laser" id="0x7f06022a" />
8      .....
9      <public type="id" name="coupon_empty_view" id="0x7f0a01f6" />
10     .....
11     <public type="layout" name="abc_action_bar_title_item" id="0x7f0d0000" />
12     .....
13     <public type="mipmap" name="asxcustom_shop_address" id="0x7f0f0000" />
14     .....
15     <public type="raw" name="silent_audio" id="0x7f110000" />
16     <public type="raw" name="beep" id="0x7f110001" />
17     <public type="raw" name="music" id="0x7f110002" />
18     .....
19     <public type="string" name="com_taobao_tae_sdk_network_not_available_message" id="0x7f1200a6" />
20     .....
21     <public type="style" name="AVLoadingIndicatorView" id="0x7f130000" />
22     .....
23 </resources>
24

```

图 3-4 /res/values/public.xml 文件内容示意

(3) 使用 Yara 匹配其全局资源 ID 的调用。

获取到无声音频文件的全局资源 ID 后, kALAD 自动编写对应 Yara 规则 (如图 3-5 所示), 在 APK 解包文件夹的字节码文件夹中使用 Yara 匹配其全局 ID 的调用, 找到存在调用的字节码文件后, 记录调用该 ID 的函数与类。

```

1  rule HexValueFields {
2      //匹配项
3      strings:
4          //匹配如下两个资源ID
5          $hex_value_0 = /0x7f0f0007/
6          $hex_value_1 = /0x7f0f0115/
7      //条件
8      condition:
9          any of ($hex_value_0, $hex_value_1)
10         //上述资源ID的任意调用都会被匹配
11 }

```

图 3-5 Yara 规则文件示例

3.3.2 拉起单像素 Activity 保活检测

图 3-6 展示了检测拉起单像素 Activity 保活行为的基本流程。KALAD 的单像素 Activity 保活行为检测流程分为三步:



图 3-6 检测单像素保活行为基本流程

(1) 从 AndroidManifest.xml 文件中筛选 APK 包含的所有 Activity。

拉起单像素 Activity 以实现保活，其核心是可启动的 Activity，故首先需要提取 APP 的所有可启动 Activity。在 Android 中，任何在 AndroidManifest.xml 文件中使用标签<activity>正确声明的组件都可以以 Activity 形式被启动，故 KALAD 首先会检查 APK 的 Manifest 文件，从中提取所有的 Activity。

(2) 遍历 Dalvik 可执行文件列表，检查 Activity 中是否存在 setAttributes 方法。由 2.3.3 节可知，开发者可以通过 setAttribute 方法，将该 Activity 的位置设置为屏幕左上角 (x=0;y=0)，宽高均设置为 1 (height=1;width=1)，从而该 Activity 设置为一个隐藏在屏幕左上角的单像素 Activity。故 KALAD 会遍历 APK 的 Dalvik 可执行文件列表，匹配检查 (1) 中所有的 Activity 是否包含函数 Landroid/view/Window;->setAttributes(Landroid/view/WindowManager\$LayoutParams;)V 的调用。如果存在该函数的调用，则会继续进行下一步的深入检查

(3) 检查特定参数赋值情况。

在找到函数 setAttributes()的调用后，需要进一步检查其中四个关键参数的调用，即 x, y, width 和 height。为了减小漏报率，KALAD 稍微放宽规则设计（因为不排除部分开发者为了规避检测，将该单像素 Activity 的位置设置为除左上角外的其他位置，故对 x 和 y 的参数设置放宽检测规则设计），当 width 和 height 两个参数均小于 2 时，判定其为单像素 Activity，记录设置这些参数的函数和类。

3.3.3 灰色前台 Service 保活检测

图 3-7 展示了检测灰色前台 Service 保活行为的基本流程。KALAD 的灰色前台 Service 检测流程分为四步：

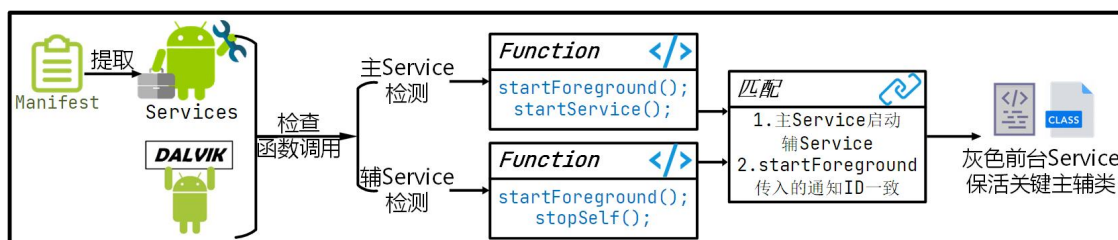


图 3-7 检测前台灰色 Service 保活行为基本流程

(1) 从 AndroidManifest.xml 文件中筛选 APK 包含的所有 Service。

启动灰色前台 Service 实现保活，其核心是可启动的 Service，故首先需要提取 APP 的所有可启动 Service。在 Android 中，任何在 AndroidManifest.xml 文件中使用标签<service>正确声明的组件都可以以 Service 形式被启动，故 KALAD 首先会检查 APK 的 Manifest 文件，从中提取所有的 Service。

(2)检查主 Service 行为。

由 2.3.4 节可知，灰色前台 Service 保活行为需要主辅两个 Service 配合实现。其中主 Service 需要先调用 startForeground()函数，再调用 StartService()函数，故 KALAD 会遍历 APK 的 Dalvik 可执行文件列表，匹配检查(1)中所有的 Service 是否调用这两个函数，如果同时存在这两个函数的调用，则会记录该 Service，以及传入 startForeground()函数的通知 ID 和传入 StartService()函数的 intent 对象的目标 Service。

(3)检查辅 Service 行为。

对于辅 Service，则需要调用 startForeground()函数和 stopSelf()函数为主 Service “打掩护”，故 KALAD 会遍历 APK 的 Dalvik 可执行文件列表，匹配检查(1)中所有的 Service 是否调用这两个函数，如果同时存在这两个函数的调用，则会记录该 Service，以及传入 startForeground()函数的通知 ID。

(4)将主 Service 和辅 Service 配对。

对检查得到的两组 Service 进行匹配，具体来说，若 ServiceA 的 startService()函数启动 ServiceB,且 ServiceB 的 startForeground()函数传入的通知 ID 与 ServiceA 的 startForeground()函数传入的通知 ID 相同，则可认为 ServiceA 与 ServiceB 是互相“打配合”实现灰色前台 Service 保活的两个 Service，并记录这对 Service。

3.4 函数调用图抽取

为对保活行为意图进行理解，仅获取实现保活行为的类和函数是不够的，还需要向上溯源，寻找借用该类进行保活以实现 APP 业务逻辑的代码和类。在 3.2 节中，已经生成了 APK 的函数调用图，故可使用该总函数调用图实现保活行为的向上溯源。但是，因为借用保活行为实现 APP 业务逻辑的代码和类，往往并不会直接调用保活行为的关键函数(以无声音频保活行为调用与使用为例，如图 3-8 所示)，而是以类为最小单位，调用保活行为的实现类，如果仅依靠函数调用图的边关系

进行溯源，容易在图结构中走入“死胡同”。

```

1  public class KeepAlivePlayMusicService extends Service {
2      @Override
3      public void onCreate() {
4          super.onCreate();
5          MediaPlayer mMediaPlayer = //使用无声音频R.raw.slient_audio进行保活
6              MediaPlayer.create(Application.getInstance(), R.raw.silent_audio);
7      }
8  }
9  //这里仅展示关键示例代码
10 public class AdActivity extends AppCompatActivity {
11     @Override
12     protected void onCreate(Bundle savedInstanceState) {
13         super.onCreate(savedInstanceState);
14         Intent intent = //启动KeepAlivePlayMusicService
15             new Intent(this, KeepAlivePlayMusicService.class);
16     }
17 }

```

图 3-8 保活实现与使用示意

为精确抽取能真实反应保活行为意图的函数调用子图，本文以 3.3 节中检测得到的保活行为核心类为出发点，设计了一种以 ClassName（类名）为层级的函数调用子图抽取方法（如图 3-9 所示）。

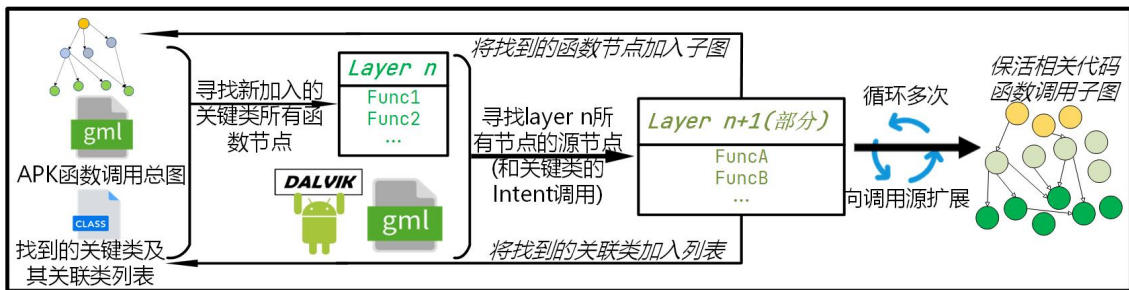


图 3-9 以 ClassName 为层级抽取函数调用图方法流程

记 APK 的函数调用总图为 G ，将 3.3 节中检测得到的保活行为核心类及其所有内部类加入集合，记为 $Class_0$ 。图 3-9 中展示的流程可分步如下步骤：

- (1) 遍历 G ，获取类名属于 $Class_0$ 的所有函数节点，加入集合，记为 $Nodes_0$ ；
- (2) 遍历 $Nodes_0$ ，在 G 中找到调用 $Nodes_0$ 中节点的所有节点（即 $Nodes_0$ 中节点的所有源节点），将这些节点加入集合，记为 $Nodes_1$ ，将这些节点所在类的类名加入集合，记为 $Class_1$ ；
- (3) 遍历 G ，获取类名属于 $Class_1$ 的所有函数节点，加入 $Nodes_1$ ，至此， $Nodes_1$ 扩展完成；
- (4) 重复上述步骤中的(2)和(3)，直至扩展到想要的层数或无法继续扩展；
- (5) 完成扩展后，遍历得到的所有 $Nodes$ 集合的所有节点，在 G 中找到所有的节点和相关联的边，构成一个新的图 $G_{extract}$ ，该子图即为所需要的可较为全面真实

地反应保活行为意图的子图;根据节点的信息,在 APK 解包得到的文件夹的 `smali/` 文件夹中,寻找函数对应的 `Smali` 形式字节码,加入集合,记为 $Smali_{G_{extract}}$ 。 $G_{extract}$ 与 $Smali_{G_{extract}}$ 将以特定方式输入给大模型,以完成最终的保活行为意图分析理解与判断。

需要额外注意的是,在第(2)步中,可能会存在“隐式可达组件”^[40]的问题,“隐式可达组件”即一个内部没有任何函数被其他类调用的 `Android` 组件类(内部函数访问修饰符全部都是 `private` 及以上,或没有类调用他的函数,这种类在静态调用链中不可达,但保留运行时可通过 `Intent` 激活的可能,如图 3-10 所示),故在遍历 G 时,无法获得 $Nodes_1$ 和 $Class_1$ 。但实际上,这些组件往往可以通过创建 `Intent` 对象从而被其他类调用,这样的类级别的关系,无法在函数调用总图 G 中表现出来。为了克服“隐式可达组件”的问题,还需在第二步中,遍历 APK 的 `Dalvik` 可执行文件列表,寻找存在的 `Intent` 初始化,并检查 `Intent` 初始化中设置的目标组件名称。若匹配到启动目标“隐式可达组件”的 `Intent` 对象,则将该类加入集合 $Class_n$,以确保可进行下一步的扩展。

```

1  public class OnePixelActivity extends Activity {
2      @Override
3      protected void onCreate(Bundle savedInstanceState) {
4          //仅展示关键单像素保活代码
5          WindowManager.LayoutParams params = getWindow().getAttributes();
6          params.width = 1;
7          params.height = 1;
8          params.x = 0;
9          params.y = 0;
10         window.setAttributes(params);
11     }
12     private void func1() {
13         System.out.println("This method is private");
14     }
15     private void unusedPrivateMethod1() {
16         System.out.println("This method is private and unused");
17     }
18 }
19

```

图 3-10 “隐式可达组件”示例

3.5 保活行为理解与意图分析

在 3.4 节中抽取得到的函数调用子图,如果一次性直接输入给大模型让其理解分析,效果并不会很好,故本文设计了一种多轮提问的方式,将子图中的内容逐

步输送给大语言模型，让其逐步理解代码，生成代码摘要，对保活行为意图进行理解与总结，并最终对保活行为良恶性进行判断（如图 3-11 所示，大语言模型提示词见附录）。

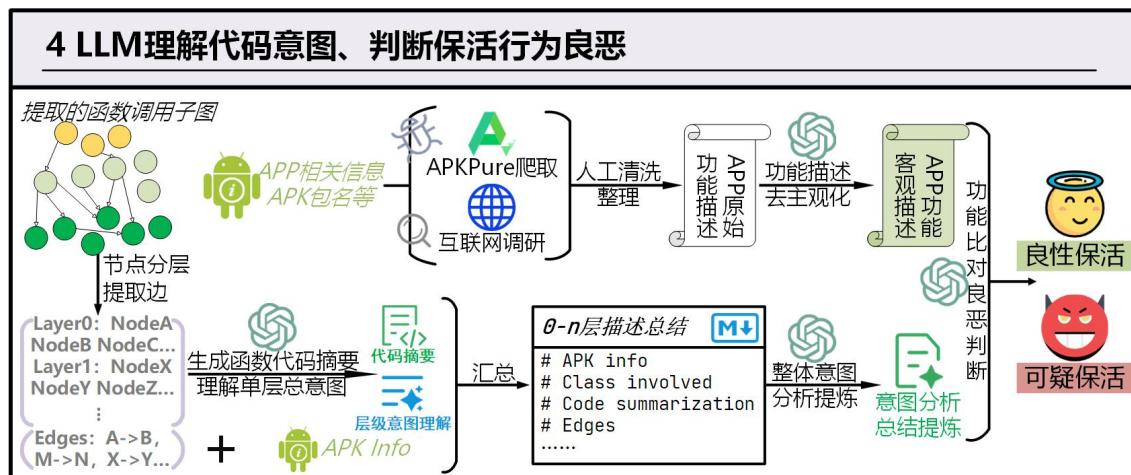


图 3-11 大模型逐步理解分析判断保活行为意图流程

KALAD 借助大模型进行保活行为意图理解分析与判断可分步如下步骤：

(1) 函数调用子图节点分层理解。

将 3.4 节中抽取得到的函数调用子图进行分层处理，将每一层的所有函数节点的 Smali 代码，依次输入给大模型，让其生成对应 Smali 代码的代码摘要，加入集合，记为 CodeSummary_n ；完成同一层的所有函数节点的代码摘要后，将同一层的所有函数的代码摘要再次输入给大模型，让其生成对这一层的整体功能描述，记为 LayerSummary_n 。

(2) 0-n 层描述汇总与整体意图提炼。

将 (1) 中生成的所有代码节点的代码摘要集合 CodeSummary_{0-n} 与 $\text{LayerSummary}_{0-n}$ 、0-n 层所有的类名集合 Class_{0-n} 、相关信息（APK 名称、可能存在的保活行为、本次子图提取的层数 n ）和节点的调用关系汇总写入 Markdown 文件中，得到 0-n 层所有节点的描述总结。将该描述总结输入给大模型，让大模型生成对这段描述总结的整体意图分析提炼，记为 Summary_{0-n} 。

(3) APP 功能描述去主观化。

从互联网上获取的 APP 功能描述信息，往往带有开发者对自己 APP 的主观褒义情感，故需要大模型对功能描述信息进行去主观化，输出对 APP 业务功能的客观描述。

(4) APP 客观功能描述与整体意图对比判断。

将(2)中得到的整体意图分析提炼 $Summary_{0-n}$ 和(3)中得到的 APP 业务功能客观描述,再次输入给大模型,让其比对保活行为意图和 APP 业务功能,分析保活行为的必要性,并判断保活行为的良恶性。

经过以上步骤,大模型可以逐步消化理解保活行为代码意图,并给出保活行为良恶性的判断。

3.6 本章小结

本章构建了一个用于检测、分析与理解 Android APK 中的保活行为的框架,框架分为预处理、保活行为检测、函数调用图抽取和保活行为理解与意图分析四个模块,在预处理阶段,KALAD 对 APK 文件进行反编译(解包),生成 APP 的完整函数调用图,提取 APP 的相关信息并解析 APK 的字节码等文件,为后续静态分析做好准备;在保活行为检测阶段,KALAD 使用针对性规则,对三种保活行为依次进行检测识别,并找出保活行为的关键函数与类;在函数调用图抽取阶段,KALAD 使用经精心设计的以类名(ClassName)为层级的函数调用图抽取算法,从保活行为的关键类出发,精确提取保活行为代码及其上下文;在保活行为理解与分析阶段,KALAD 借助大模型,使用多轮提问方式,让大模型逐步理解保活行为相关代码,并最终给出保活行为的良恶判断。

通过以上步骤,KALAD 可系统化、自动化地从 APK 文件中逐步提取特定保活行为特征与代码,抽取保活行为相关函数调用子图,利用大模型对保活行为进行理解、分析与良恶判别。

4 实验结果与分析

本节进行了大量实验,通过回答如下三个问题对 KALAD 的有效性和性能进行了评估,并对三种保活行为的在野特性进行了一定深入分析:

RQ1: KALAD 检测这三种保活行为及其良恶的有效性如何?

RQ2: KALAD 的性能如何? 有哪些因素显著影响其性能?

RQ3: 三种保活行为的在野特性如何?

4.1 实验设置

4.1.1 实验环境

本节的实验环境软硬件信息如表 4-1 所示。实验使用 Python 作为主要编程语言,大语言模型选用 GPT-3.5。

表 4-1 实验环境软件与硬件信息

硬件/软件	型号与描述信息
CPU	Apple M1 Pro (6 Performance and 2 Efficiency)
RAM	16GB
MacOS Version	Sequoia 15.4.1
LLM	GPT-3.5 turbo
Python	3.11.11
Androguard	4.1.2
APKTool	2.11.1

4.1.2 实验数据集

本文构建了两个数据集(包括 $Data_{test}$ 和 $Data_{wild}$)用于回答三个研究问题 RQ1~RQ3。为回答 RQ1~RQ2,通过与华为安全部合作,本文从华为应用市场收集一批保活可疑性较高的 APP 样本,经过筛选,最终留下 252 个 APK 样本,构成数据集 $Data_{test}$;为回答 RQ3,本文从 XLDH 数据集^[41]中采集 971 个 APK 样本,构成数据集 $Data_{wild}$ 。

除此之外,本文还编写 Python 爬虫脚本,从应用市场(APKPure)上爬取 APK 相关的功能描述,并人工清洗爬取得到的描述信息(删除误爬取的更新日志、删除重复描述等),分别构建了与 $Data_{test}$ 和 $Data_{wild}$ 配套的两个数据集 $Desc_{test}$ 和 $Desc_{wild}$ 。

4.1.3 评价指标

(1) 保活行为检测及其良恶的有效性评估。

本文使用准确率 (Accuracy)、精确率 (Precision)、召回率 (Recall) 和 F1 值 (F1 Score) 评估 KALAD 检测三种保活行为及其良恶的有效性。准确率：被正确预测的样本比例；精确率：预测的正类中，正确预测为正类的样本比例；召回率：实际的正类中，被正确预测为正类的样本比例；F1 值：精确率与准确率的调和平均数，衡量分类器在不平衡数据集下的性能。具体而言，对于一个二分类任务，分类结果存在如表 4-2 所示的四种情况：

表 4-2 二分类任务混淆矩阵

	N(Negative)	P(Positive)
F(False)	FN (预测错误, 预测为 N, 实际为 P)	FP (预测错误, 预测为 P, 实际为 N)
T(True)	TN (预测正确, 预测为 N, 实际为 N)	TP (预测正确, 预测为 P, 实际为 P)

如此，准确率、精确率、召回率和 F1 值可由如下公式得到：

$$\text{Accuracy} = \frac{\text{TN} + \text{TP}}{\text{FN} + \text{FP} + \text{TN} + \text{TP}} \quad (4-1)$$

$$\text{Precision} = \frac{\text{TP}}{\text{FP} + \text{TP}} \quad (4-2)$$

$$\text{Recall} = \frac{\text{TP}}{\text{FN} + \text{TP}} \quad (4-3)$$

$$\text{F1 Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (4-4)$$

使用以上指标，可较为全面的衡量 KALAD 的有效性。

(2) KALAD 性能评估。

本文通过在 KALAD 代码中打点插桩计时，通过收集多段计时结果，并统计 KALAD 运行时可能对性能有影响的部分数据，对显著影响 KALAD 性能的多种因素进行了较为全面的评估。具体来说，显著影响 KALAD 分析一个 APK 用时的主要因素有如下两点：

1. APK 中存在的保活行为数量，因 KALAD 对每一处保活行为都会进行 0~2 层的函数调用子图扩展和借助大模型进行理解分析的工作，会影响运行

KALAD 的运行时间;

2. 使用 Androguard 进行预处理以获取 APK 的 Dalvik 可执行文件列表和 APK 相关信息 (即 AnalysisAPK() 函数) 的时间, 这部分时间可能会受 APK 文件大小或是 APK 包含的函数个数影响;

对于 2, 本文将每个 APK 的相关特征数据与对应分析耗时进行配对, 绘制散点图以观察两者之间的趋势关系。随后, 利用简单线性回归对数据进行拟合, 尝试挖掘各因素与分析耗时之间的线性相关性。在简单线性回归中, 对于一组 n 个样本数据点 $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$, 希望通过一个线性函数来刻画自变量 x 与因变量 y 之间的关系。其基本形式如式 4-5 所示, 其中 β_0 是截距, β_1 是斜率, ε 是误差值。

$$y = \beta_0 + \beta_1 x + \varepsilon \quad (4-5)$$

使用最小二乘法 (OLS) 对截距 β_0 和斜率 β_1 进行计算的公式如式 4-6 和式 4-7 所示, 其中 $\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$ 为自变量的均值, $\bar{y} = \frac{1}{n} \sum_{i=1}^n y_i$ 为因变量的均值。

$$\beta_1 = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2} \quad (4-6)$$

$$\beta_0 = \bar{y} - \beta_1 \bar{x} \quad (4-7)$$

拟合完成后, 使用评估指标决定系数 (R^2) 和均方误差 (MSE) 来判断拟合质量, 其中, R^2 和 MSE 由式 4-8 和式 4-9 和式计算得出, 其中 $\hat{y}_i$ 为拟合曲线预测值:

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2} \quad (4-8)$$

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (4-9)$$

通过以上方式拟合与评估, 可以找出对 KALAD 运行性能相关性较强的因素。

对于 1, 由于其离散特性不宜直接进行拟合分析, 本文采用分组对比的方式, 将 APK 按所包含的保活行为数量进行分类统计, 并计算每类 APK 的平均分析用时。通过对比不同类别下的平均耗时差异, 可以观察保活行为数量对 KALAD 运行时间的具体影响。

4.2 实验结果

本节将依照 RQ1~RQ3 的顺序，依次介绍实验方法，并分析得到的实验结果。

4.2.1 RQ1: KALAD 的有效性

(1) KALAD 检测三种保活行为的有效性。

实验方法：为评估 KALAD 检测三种保活行为的有效性，本文将 $Data_{test}$ 数据集的一共 252 个 APK 样本输入 KALAD 中执行静态分析，完成分析后，对这 252 个样本全部进行手动检查以确定它们是否具有（或不具有）保活行为。

实验结果：经过 KALAD 的静态分析，一共检测出 239 个 APK 中存在 353 处保活行为（一个 APK 中可能存在一种保活行为的多个实现、也有可能存在多种保活行为）。表 4-3 展示了三种保活行为的检测结果。由表可知，在 $Data_{test}$ 的 252 个数据集中，对于无声音频与单像素 Activity 保活行为，KALAD 有 100% 的准确率，且其精确率、召回率和 F1 值也为 100%；对于灰色前台 Service 保活行为，KALAD 一共成功检测出了 84 个样本，漏掉了 2 个样本，在经过手动分析后发现，漏掉的两个样本中存在的灰色前台 Service 保活行为，均来自字节跳动的 SDK 中的保活行为（类名为 `com.bytedance.mira.am.KeepAlive`），其保活实现代码经过了多轮逻辑跳转与代码混淆，故而逃过了 KALAD 的检测。但总体来看，KALAD 对这三种保活行为的识别均达到了近乎 100% 的识别准确率。

表 4-3 KALAD 检测三种保活行为的结果

保活行为	KALAD 检测结果	手动检查结果	差值	准确率
无声音频	28	28	0	100%
单像素 Activity	195	195	0	100%
灰色前台 Service	84	86	漏检 2 个	99.21%

(2) KALAD 使用大模型进行保活行为良恶判断的有效性。

实验方法：为评估 KALAD 使用大模型进行保活行为良恶判断的有效性，本文将 $Data_{test}$ 数据集中存在的 353 处保活行为分别经过 0、1、2 次函数调用子图扩展后，使用大模型进行处理分析，完成后，再对这 353 处保活行为全部进行手动检查以确定它们的良恶性。

实验结果：经过大模型的分析理解与手动检查，得到了对这 353 处保活行为的分类结果汇总。表 4-4 展现了扩展层数从 0 到 2 的分类结果。由表可知，KALAD

在 $Data_{test}$ 中找到的 353 处保活行为，一共有 317 处良性保活行为和 36 处可能的恶意保活行为（或可疑的保活行为）。从结果来看，KALAD 使用大模型能够识别大部分的良性与可能的恶意保活行为，在使用不同扩展层数（0~2）进行函数调用子图扩展的情况下，KALAD 均具有超过 80%的准确率。此外，随着扩展层数的上升，大模型对代码上下文的理解更充分，整体判断性能（准确率、精确率、召回率和 F1 值）也随之上升。具体来说，当扩展层数为 0（即仅分析保活点本身代码）时，模型判断准确率为 84.14%；当扩展层数提升至 1 时，准确率提升至 87.72%；进一步扩展至 2 层时，准确率达到 88.39%，加权精确率为 0.89，加权召回率为 0.88，加权 F1 值为 0.89，表明大模型对保活行为上下文的理解能力在一定程度上随扩展层数的增加而增强，从而提高了分析理解判断能力。

表 4-4 KALAD 使用大模型判断保活行为良恶结果汇总

扩展层数	判良实良	判良实恶	判恶实良	判恶实恶	实良	实恶	准确率
0	283	22	34	14			84.14%
1	292	18	25	18	317	36	87.72%
2	294	18	23	18			88.39%

综上所述，KALAD 在检测与分析 Android 应用中的保活行为方面展现出了良好的有效性。在保活检测方面，KALAD 可以成功识别出数据集中的三种主流保活行为（即无声音频、单像素 Activity，灰色前台 Service），其中前两种保活行为的检测准确率达到了 100%，而对于较为复杂的灰色前台 Service 保活行为，仍然实现了 99.21%的高准确率，展现出其在静态分析下的强大保活行为识别能力。在保活行为良恶判定方面，KALAD 借助大语言模型进行语义理解与分类，在不同的函数调用子图扩展层数下均保持了超过 80%的准确率，且随着扩展层数的增加，性能稳步提升，表明 KALAD 具备较好的保活行为理解分析与判断能力，对保活行为的研究提供有效支持。

4.2.2 RQ2: KALAD 的性能

实验方法：为评估 KALAD 的性能，本文通过在代码中增加插桩计时部分，统计总用时与部分模块用时（包括 Androguard 执行预处理的时间、保活检测时间，每一层级的大模型用时），并统计 KALAD 运行时可能对性能有影响的部分数据（包括 APK 文件大小、APK 包含函数总数、每一层的节点个数），对运行检测 $Data_{test}$

数据集集中的 252 个 APK 时的性能进行了分析。

实验结果：经过插桩计时与统计分析，实验结果如下所示：

(1) 针对 $Data_{test}$ 数据集集中的 252 个 APK 样本，KALAD 在执行完整静态分析和保活行为理解判断过程中，其综合分析处理平均用时为 437.51 秒/APK，图 4-1 展示了 APK 中使用的保活行为数量与 KALAD 综合分析处理时间之间的关系。整体趋势表明：保活行为数量的增加对分析时间具有显著影响。可以看到，当 APK 中不包含保活行为时，KALAD 综合分析处理时间平均为 115.35 秒，而随着 APK 使用保活行为数量的增加，KALAD 的综合分析处理时间逐步增加，尤其是当保活行为数量达到 6 个甚至更多时，处理时间显著增加到 1200 秒以上。这表明保活行为的复杂性和密度直接影响 KALAD 分析所需的处理资源和时间开销。

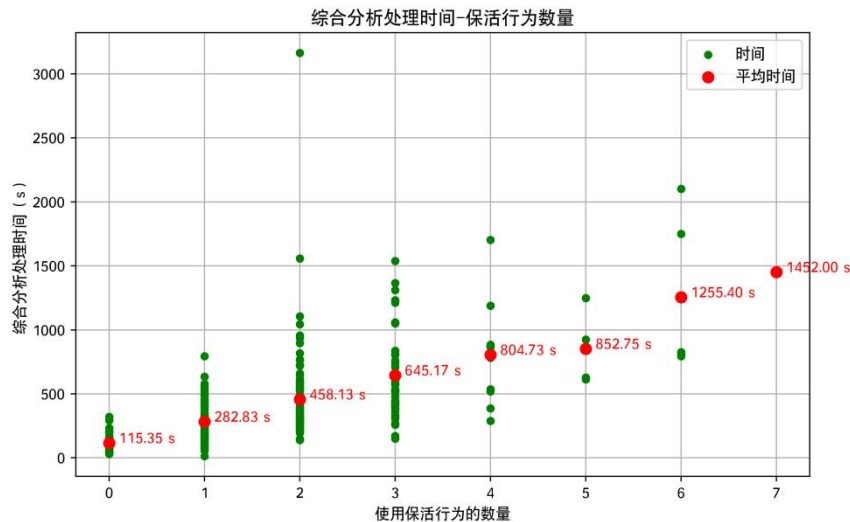


图 4-1 APK 包含保活行为数量与 KALAD 分析时间关系散点图

(2) 针对 $Data_{test}$ 数据集集中的 252 个 APK 样本，KALAD 在预处理阶段（调用 Androguard 的 AnalysisAPK() 函数以获取 APK 的 Dalvik 可执行文件列表和 APK 相关信息）的平均耗时为 186.28 秒。为进一步探究影响该阶段处理时间的关键因素，图 4-2 展示了 APK 文件大小（左）和 APK 包含的函数数量（右）分别与 Androguard 预处理用时的关系。通过简单线性回归拟合分析可以观察到，APK 包含的函数数量对处理时间的影响更加显著。在图 4-2 的右图中，拟合曲线斜率较大，点分布更接近拟合直线，说明 APK 包含的函数数量越多，Androguard 的分析处理时间越长，两者呈现更强的正相关关系。而相比之下，图 4-2 的左图中，APK 文件大小与处理时间之间的拟合关系较弱，点分布较为离散，说明 APK 体积虽有一定影响，但

并非主要决定因素。

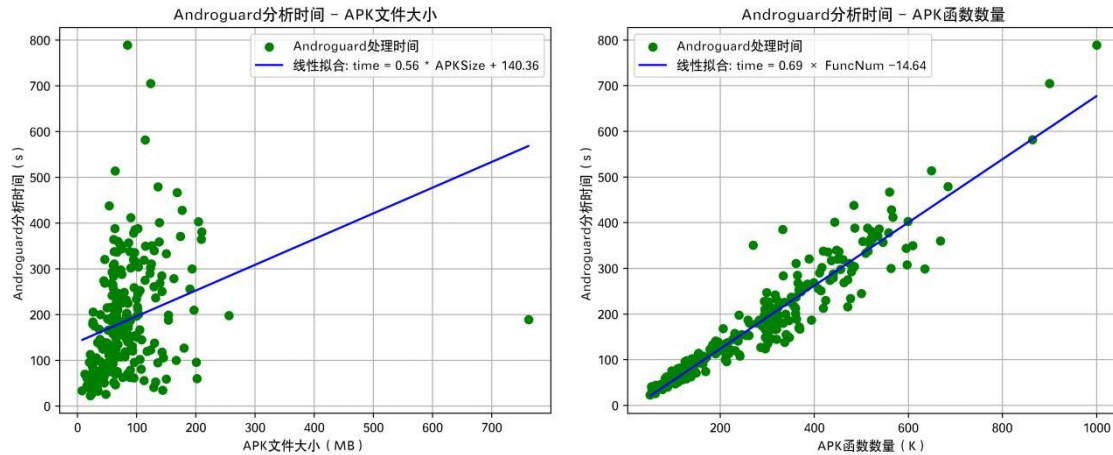


图 4-2 APK 大小与函数数量与 Androguard 处理时间关系图

除此之外，为了更客观地比较 APK 文件大小与 APK 包含函数数量这两个因素对 Androguard 预处理时间的解释能力，表 4-5 进一步列出了对应线性拟合模型的评估指标。由表可以看出，基于 APK 包含函数数量的线性拟合展现出更高的拟合优度，其 $R^2=0.8984$ ，MSE 为 1440.7，均显著优于基于 APK 文件大小的线性拟合。

表 4-5 APK 大小与函数数量线性拟合 Androguard 处理时间的评估结果

	APK 文件大小	APK 包含函数数量
拟合曲线	$\text{Time} = 0.56 \times \text{APKSize} + 140.36$	$\text{Time} = 0.69 \times \text{FuncNum} - 14.64$
R^2	0.0831	0.8984
MSE	13005.97	1440.7

综上所述，KALAD 在进行保活行为检测和理解分析的过程中，其整体用时受到多种因素的影响。其中分析时长和保活行为的数量呈正相关，说明保活行为的复杂性显著增加了他的分析负担；而在预处理阶段，APK 包含的函数数量对 Androguard 处理时间的影响显著强于 APK 文件大小。整体而言，KALAD 的性能在处理规模化 APK 数据时表现出了较为良好的可解释性，为后续优化和扩展提供了明确方向。

4.2.3 RQ3：三种保活行为在野特性

实验方法：为研究三种保活行为的在野特性，本文将 $Data_{wild}$ 数据集中的 971 个 APK 输入 KALAD 中执行检测分析处理，并对分析检测出的结果进行手动分析检查，以理解其保活行为意图。

实验结果: 经过 KALAD 的检测分析处理, 一共检测出 $Data_{wild}$ 数据集中的 971 个 APK 中, 存在 4 个 APK 有无声音频保活行为, 4 个 APK 有单像素 Activity 保活行为, 5 个 APK 有灰色前台 Service 保活行为。经过对这些 APK 的手动分析, 得到以下结论:

(1) 4 个存在无声音频保活行为的 APK 中, 其中有 2 个为运动健康记录 APP, 经过手动检查发现, 这 2 个 APP 使用无声音频保活行为, 主要是用于在运动健身过程中, 持续记录运动状态或运动地理位置, 从而提升用户的运动健身体验; 另外 2 个均为视频制作剪辑 APP, 经过手动检查发现, 这 2 个 APP 使用无声音频保活行为则主要是用于在后台中持续处理视频编辑工作。总体来看, 暂未发现这 4 个 APP 存在恶意利用保活行为的迹象。

(2) 4 个存在单像素 Activity 保活行为的 APK 中, 其中 2 个为运动健康记录 APP (与(1)中存在的无声音频保活行为的 APK 存在 1 个 APK 重合), 经过手动检查发现, 这 2 个 APP 的单像素保活行为主要是用于在运动健身过程中, 持续记录用户的运动状态, 从而提升用户的运动健身体验; 1 个用于检测手相的工具 APP 则是被检测到友盟的推送服务^⑭SDK (Software Development Kit, 软件开发工具包) 中存在单像素保活行为, 经过手动检查, 确定其是为了及时向用户推送通知; 1 个用于海外访问国内互联网的 APP 则是被检测到含有的 Xinstall^⑮的 SDK 中存在单像素保活行为, 经手动分析与互联网调研, 推测该 SDK 可能会收集用户的隐私信息, 以便用于渠道更新或用户拉新分析, 可能存在一定的恶意性。

(3) 5 个存在灰色前台 Service 保活行为的 APK 中, 其中 1 个为监控摄像头视频传输 APP, 被检测到含有 UPU (Universal Postal Union, 万国邮政联盟) 的 DPS 推送服务, 经过手动检查, 该保活行为用于实时推送远端设备异常事件 (如家庭摄像头的断电或视频丢失等、企业安防的非法入侵等) 到用户手机; 1 个外卖 APP 被检测到含有腾讯的开源热修复框架 Tinker 的下载 Service, 经过手动检查和互联网调研分析, 该保活行为用于支持 DEX、库和资源的热更新; 1 个锁屏壁纸 APP 被检测出使用灰色前台 Service 保活行为以支持锁屏壁纸的自定义更换功能; 1 个应用克隆 APP 被检测实现灰色前台 Service 以最大程度保持虚拟环境的持久性; 1

^⑭ 来源: <https://devs.umeng.com/sdk>

^⑮ 来源: <https://www.xinstall.com/>

个用于精确找物的 APP 被检测出使用保活行为持续进行蓝牙信标扫描,用于在靠近物品时的近距离导航。综合来看,暂未发现这些 APP 恶意利用灰色前台 Service 保活的行为。

总结来看, KALAD 在对 *Data_{wild}* 数据集中的 971 个 APK 样本的检测中, 成功识别若干保活行为的应用实例, 经过人工检查, 大多数检测结果均与应用的功能场景较为相关, 如运动健康 APP 为持续记录运动状态, 监控摄像头视频传输 APP 为及时提醒用户, 均属于合理范围内使用。个别 APK 所携带的第三方收集用户数据 SDK 则可能存在过度收集用户隐私数据的恶意行为, 值得关注。总体而言, 在野的 APP 使用保活行为, 大部分都是在业务范围内的合理使用, 个别可能存在的恶意行为, 则需要进行更加严格的合规性审查。

4.3 本章小结

本章围绕 KALAD 的有效性与性能展开了系统性的评估实验, 旨在回答三个关键问题: KALAD 检测三种保活行为及其良恶的有效性 (RQ1), KALAD 的性能与影响因素 (RQ2) 以及保活行为在真实 APK 应用中的在野特征表现 (RQ3)。

在有效性方面, KALAD 能较高准确率的检测出无声音频、单像素 Activity 和灰色前台 Service 保活行为, 并借助大模型实现了较高准确率的保活行为良恶判断能力。

在性能方面, KALAD 的整体分析时间受 APK 包含的保活行为数量影响显著, 预处理阶段中 APK 包含的函数数量则是主要影响因素, 线性关系明显。

在在野特征方面, 通过对 971 个 APK 的检测分析, 结果表明, 在野的 APP 使用保活行为, 大部分都是在业务范围内的合理使用, 仅个别存在可疑行为。

总体而言, KALAD 在保活行为分析与理解任务中展现出良好的准确性、性能表现和实际适用性。

5 总结与展望

5.1 本文小结

本文围绕 Android 应用保活行为的检测与意图分析展开研究,针对当前学术界对 Android 保活行为研究不足的问题,设计并实现了一种基于层级函数调用图的保活识别与意图分析框架——KALAD。使用静态分析、规则检测和大语言模型理解等技术手段,KALAD 能够系统化、自动化地完成三种特定保活行为(后台播放无声音频保活、拉起单像素 Activity 保活、灰色前台 Service 保活)的检测与意图分析。

本文首先梳理了 Android 保活行为的背景与研究意义,分析了国内外研究现状,并指出当前研究在检测工业界中使用的特定保活手段、处理庞大代码规模以及理解代码真实意图等方面存在的挑战。在此基础上,本文提出了 KALAD 框架的整体设计,并逐步详细介绍了预处理、保活行为检测、函数调用图抽取、和保活行为理解分析四个模块的具体实现。

然后,本文通过实验验证了 KALAD 的有效性与性能。对收集自华为应用市场的 252 个保活高可疑度样本的实验结果表明,KALAD 能以几乎 100%的准确率检测出三种常见的保活行为,并借助大语言模型实现较高的准确率和良恶判断能力。此外,本文还通过对 971 个在野真实 APK 样本的分析,揭示了保活行为的在野特征。

最后,本文总结了 KALAD 的创新与不足,并对未来的深入研究提出了展望。本文为 Android 应用保活行为的检测与意图分析提供了新的技术思路,对提升智能手机用户体验,捍卫健康的移动应用生态有重要意义。

5.2 创新点与贡献

本文的主要创新点与贡献如下:

(1) 本文构建了一个基于层级函数调用图的保活行为检测与分析理解框架,通过此理解框架,本文首次对工业界三种主流保活行为(无声音频、单像素 Activity、灰色前台 Service)进行了大规模实证研究,揭示了这些技术在真实应用中的分布特征和使用意图。研究发现大多数保活行为与应用功能需求相关,但也识别出个

别可能存在隐私风险的 SDK 实现，为行业监管提供了重要参考。

(2) 本文提出了一种以类名 (ClassName) 为层级的函数调用图抽取算法，能从庞大的 APP 函数调用图中精确定位保活行为相关代码及其上下文。该方法通过逐层扩展函数调用关系，较好地克服了直接在函数调用图中进行代码溯源容易陷入“死胡同”的问题，为框架进行保活行为的检测与分析提供了更完整的代码上下文，实验结果表明，框架借助这种算法区分良恶保活行为的有效性随着扩展层数的上升有一定上升。

5.3 不足与展望

尽管本文取得了一定的成果，但仍存在以下不足，期望能在未来的继续深入研究工作中进一步完善：

(1) 无法分析加壳 APK。KALAD 的所有检测分析流程都依赖于对 APK 的静态分析，然而，经过加壳保护的 APK 文件，简单的静态分析工具无法分析所有的 DEX 文件，这限制了工具在真实工业场景中的应用范围。未来期望可以研究加入对抗加壳技术的方法，以提升对加壳应用的检测能力。

(2) 检测覆盖范围的局限性。当前 KALAD 仅针对三种典型的保活行为进行检测，而工业界中真实使用的保活技术仍然在不断演进。未来需要持续跟踪新型保活手段，以扩展可检测保活行为范围。

(3) 大模型理解能力的优化。当前大模型对代码的理解仍然存在一定误差，尤其是当代码量较多时，理解分析能力欠佳。未来需要探索更多的方法以优化大模型的理解能力，或是结合传统神经网络、传统符号执行程序分析技术以提升理解精度。

参考文献

- [1] M. Ghafari, P. Gadiant, O. Nierstrasz. Security smells in Android. CoRR, 2020, abs/2006.01181
- [2] 李豁然, 刘譞哲, 梅俏竹 等. 基于细粒度数据的智能手机续航时间预测模型. 软件学报, 2021, 32(10): 3219-3235
- [3] S. Kang, H. Choi, S. Park, et al. Fire in your hands: Understanding thermal behavior of smartphones. The 25th Annual International Conference on Mobile Computing and Networking. Los Cabos, Mexico; Association for Computing Machinery. 2019: Article 13.10.1145/3300061.3300128
- [4] H. Zhou, H. Wang, Y. Zhou, et al. Demystifying diehard Android apps. The 35th IEEE/ACM International Conference on Automated Software Engineering. 2020: 187-198. DOI: 10.1145/3324884.3416637
- [5] 颜宏冰. 安卓应用中资源滥用行为检测方法研究[硕士学位论文]. 合肥: 中国科学技术大学, 2019
- [6] Y. Yan, Z. Li, Q. A. Chen, et al. Understanding and detecting overlay-based Android malware at market scales. The 17th Annual International Conference on Mobile Systems, Applications, and Services. 2019: 168-79.10.1145/3307334.3326094
- [7] Y. Shao, R. Wang, X. Chen, A. M. Azab, Z. M. Mao. A lightweight framework for fine-grained lifecycle control of Android applications. The Fourteenth EuroSys Conference 2019, Dresden, Germany, Association for Computing Machinery, 2019: Article 31. DOI: 10.1145/3302424.3303956
- [8] A. Carette, M. A. Ait Younes, G. Hecht, et al. Investigating the energy impact of Android smells. in: IEEE International Conference on Software Analysis, Evolution, and Reengineering, 2017. DOI: 10.1109/SANER.2017.7884614
- [9] W. Oliveira, R. Oliveira, F. Castor. A study on the energy consumption of Android app development approaches. The 14th International Conference on Mining Software Repositories, Buenos Aires, Argentina, IEEE Press, 2017: 42 - 52. DOI: 10.1109/MSR.2017.66

- [10] S. Chowdhury, S. Nardo, A. Hindle, Z. Jiang. An exploratory study on assessing the energy impact of logging on Android applications. *Empirical Software Engineering*, 2018, 23. DOI: 10.1007/s10664-017-9545-x
- [11] C. Gao, M. Cai, S. Yin, G. Huang, H. Li, W. Yuan, et al. Obfuscation-resilient Android malware analysis based on complementary features. *IEEE Transactions on Information Forensics and Security*, 2023, 18: 5056-5068
- [12] J. Lee, H. Jang, S. Ha, et al. Android malware detection using machine learning with feature selection based on the genetic algorithm. *Mathematics*, 2021, 9(21): 2813
- [13] M. C. Ipek, S. Sen. Explainable Android malware detection and malicious code localization using graph attention. *arXiv preprint arXiv:2503.07109*, 2025
- [14] Q. Xie, Q. Gong, X. He, et al. Trimming mobile applications for bandwidth-challenged networks in developing regions. *IEEE Transactions on Mobile Computing*, 2021, 22(1): 556-573
- [15] H. Kang, J.-W. Jang, A. Mohaisen, et al. Detecting and classifying Android malware using static analysis along with creator information. *International Journal of Distributed Sensor Networks*, 2015, 11(6): 479174
- [16] L. Onwuzurike, E. Mariconti, P. Andriotis, et al. MaMaDroid: Detecting Android malware by building Markov chains of behavioral models (extended version). *ACM Transactions on Privacy and Security*, 2017, 22
- [17] J. Yang, Z. Zhang, H. Zhang, et al. Android malware detection method based on highly distinguishable static features and DenseNet. *PLOS ONE*, 2022, 17(11): e0276332
- [18] A. Muzaffar, H. Hassen, H. Zantout, et al. DroidDissector: A static and dynamic analysis tool for Android malware detection. in: *International Conference on Applied CyberSecurity (ACS)*, Dubai, United Arab Emirates, 29 Apr 2023. *Lecture Notes in Networks and Systems*, vol 760. Springer, 2023: 3-9
- [19] W. Sun, Y. Miao, Y. Li, et al. Source code summarization in the era of large language models. 2025 *IEEE/ACM 47th International Conference on Software Engineering (ICSE)* 26-03 May 2025: 419-431

- [20] K. Yim, I. Malchev, A. Hsieh, et al. TREBLE: Fast software updates by creating an equilibrium in an active software ecosystem of globally distributed stakeholders. *ACM Transactions on Embedded Computing Systems (TECS)* 2019. 18(5s): 1-23
- [21] M. Jaiswal. Android the mobile operating system and architecture. *SSRN Electronic Journal*, 2018, 6: 514-525
- [22] M. Goel, G. Singal. Android OS case study. *ArXiv* 2021 Vol. abs/2104.09487
- [23] D.-H. You, B.-N. Noh. Android platform based Linux kernel rootkit. *The 6th International Conference on Malicious and Unwanted Software* 18-19 Oct. 2011
- [24] Y. Gan, W. Hu, Y. Wang. Design and optimization of Camera HAL layer based on Android. *Smart Computing and Communication: 4th International Conference, SmartCom 2019, Birmingham, UK, October 11 – 13, 2019, Proceedings. Birmingham, United Kingdom; Springer-Verlag. 2019: 296 – 303.10.1007/978-3-030-34139-8_29*
- [25] S. L. Sanna, D. Soi, D. Maiorca, et al. A risk estimation study of native code vulnerabilities in Android applications. *Journal of Cybersecurity*, 2024, 10(1): tyae015
- [26] R. Yadav. Performance analysis for Android runtime environment. *The Fifth International Conference on Communication Systems and Network Technologies* 4-6 April 2015
- [27] B. Kreyssig, T. Riom, S. Houy, et al. Deserialization gadget chains are not a pathological problem in Android: an in-depth study of Java gadget chains in AOSP. *CoRR* 2025 Vol. abs/2502.08447
- [28] H. Lee, J. Park, U. Lee. A systematic survey on Android API usage for data-driven analytics with smartphones. *ACM Comput. Surv.* 2022 Vol. 55 Issue 5 Pages Article 104
- [29] J. Yan, S. Zhang, Y. Liu, et al. A comprehensive evaluation of Android ICC resolution techniques. *The 37th IEEE/ACM International Conference on Automated Software Engineering* Rochester, MI, USA; Association for Computing Machinery. 2023: Article 1.10.1145/3551349.3560420
- [30] J. Samhi, A. Bartel, T. Bissyandé, et al. RAICC: Revealing atypical inter-component communication in Android apps. *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*, 2021, IEEE Pages: 1398-1409

- [31] B. Panigrahi, N. Nagarajan, K. Prasad, et al. Novel nature-inspired optimization approach-based SVM for identifying the Android malicious data. *Multimedia Tools and Applications*, 2024, 83: 1-19
- [32] T. Wang, Y. Xu, X. Zhao, et al. Android malware detection via efficient application programming interface call sequences extraction and machine learning classifiers. *IET Software*, 2022, 17(4): 348-361
- [33] E. J. Chikofsky, J. H. Cross. Reverse engineering and design recovery: a taxonomy. *IEEE Software*, 1990, 7(1): 13-17
- [34] J. Han, S. Lee. Performance improvement of Linux CPU scheduler using policy gradient reinforcement learning for Android smartphones. *IEEE Access*, 2020, 8: 11031-11045
- [35] 珠海格力电器股份有限公司. 音乐播放应用保活处理方法、系统、存储介质及电子设备: 中国, 发明专利申请, CN112162783A. 2021
- [36] M. Chen, J. Tworek, H. Jun, et al. Evaluating large language models trained on code. *ArXiv 2021 Vol. abs/2107.03374*
- [37] Y. Wang, H. Le, A. D. Gotmare, et al. CodeT5+: Open code large language models for code understanding and generation. 10.48550/arXiv.2305.07922.
- [38] Z. Feng, D. Guo, D. Tang, et al. CodeBERT: A pre-trained model for programming and natural languages. Online, F November, 2020. Association for Computational Linguistics.
- [39] J. Pinochet. Graphic relation between amplitude and sound intensity level. *The Physics Teacher*, 2021 , 59(6): 467-9
- [40] L. Li, A. Bartel, T. F. Bissyandé, et al. IccTA: Detecting inter-component privacy leaks in Android apps. in: 2015 IEEE/ACM 37th IEEE International Conference on Software Engineering (ICSE), Florence, Italy, 16-24 May 2015, IEEE, 2015
- [41] J. Wang, Y. Xiao, X. Wang, et al. Understanding malicious cross-library data harvesting on Android. *The 30th USENIX Security Symposium (USENIX Security 21)*, August 2021, USENIX Association

附录

3.5 节中, 调用大模型的提示词如附录表 1~附录表 5 所示:

表 1 大模型生成函数代码摘要

提示词部分	提示词
任务描述	Analyze the provided JAVA/Smali code snippet or function sign and generate a concise, high-level summary of its functionality(Output is one line without line feeds).Focus on the code's purpose, not implementation details.
输出要求	Follow these guidelines: 1. Describe the primary actions (e.g., data manipulation, API calls). 2. Highlight key methods or system interactions (e.g., onCreate, network requests). 3. Avoid technical jargon; use plain language. 4. Keep the summary under 3 bullet points.
输出示例	Initializes UI components and binds event listeners.Validates user input against a regex pattern.Sends encrypted data to a remote API.
输入部分	My Input: {Functions' byte code in smali}

表 2 大模型生成层级意图总结

提示词部分	提示词
任务描述	Analyze the provided JSON layer definition (including class_name_list and func_node_list) and determine its primary purpose in the APK's architecture. Follow these steps: 1. Identify Key Patterns: (1) Map relationships between class names (e.g., KeepAlive, InnerService) and functional nodes. (2) Highlight recurring themes like service management, background persistence, or resource tracking.
输出要求	2. Synthesize Node Roles: (1) For each func_desc, extract the core action (e.g., creates inner service, logs execution details). (2) Cluster related actions to identify overarching goals. 3. Define Layer Purpose: Summarize the layer's role in 1-2 sentences, avoiding implementation details. (Focus on system-level behavior, not individual method logic.)
输出示例	Example Output: Implements a background service keep-alive mechanism to bypass system restrictions.Uses KeepAlive and InnerService classes to maintain persistent processes.Tracks service execution via conditional checks and logging.
输入部分	My Input: {Functions' byte code summary}

表 3 大模型整体意图分析提炼

提示词部分	提示词
任务描述	Based on the provided Markdown documentation, identify the PRACTICAL FUNCTION sustained by the APK's keep-alive mechanism.
输出要求	Follow strictly: 1. Functional Purpose Only: State the real-world service/feature enabled by background persistence. (e.g., "Background audio streaming for music playback"), NOT technical implementation details. 2. Required Analysis Path (1) Identify core classes (e.g., MediaPlayerService → audio; e.g.2, com.vivo.push.sdk.service.LinkProxyActivity → Message Push). (2) Map to Android permissions/APIs (e.g., BLUETOOTH_CONNECT → device control). (3) Infer user-facing scenarios (fitness track, ads, surveillance, etc.).
输出示例	Here are some succinct examples that you can add to as you see fit, but don't go beyond 3 sentences: Bad Response (avoid): "Uses JobScheduler to restart terminated processes." (Technical mechanism) Good Response: "Background audio streaming for music playback"
输入部分	My Input: {Keep-Alive Behavior's Markdown summary}

表 4 大模型对 APP 功能描述去主观化

提示词部分	提示词
任务描述	Convert the given Chinese app description into a neutral, objective English summary. Your Task: 1. Output only the objective summary in English(No more than 3 sentences). 2. Use bullet points or a paragraph; avoid markdown. 3. No use of exaggerated or promotional terminology 4. Exclude claims about user outcomes (e.g., '降低发病率' → remove). 5. Focus on WHAT the app does, not WHY it claims to do it."
输出要求	following these steps: 1. Remove Subjective Language: (1) Delete Exaggerated descriptors (e.g., '顶尖', '顶级', '尖端') and marketing terms (e.g., '致力于成为...专业的', '先进的'). (2) Delete sentences related to users, avoid value judgments (e.g., '提升感受', '帮助用户降低...'). 2. Retain Core Functions:

	(1) Focus on technical actions (e.g., data collection, analysis). (2) Preserve service offerings (e.g., consultations, assistance). 3. Restructure Objectively: (1) Use passive voice or neutral phrasing (e.g., 'provides X' instead of 'helps users achieve X'). (2) Prioritize conciseness; avoid explanations of benefits.
输出示例	Example Input: 启康保致力于成为中国专业的个性化健康服务平台；通过健康数据的多样化采集存储、疾病趋势分析、三级实时预警体系以及先进的疾病早筛工具，帮助用户降低重大疾病发病率；管理和延缓慢性疾病的发生发展。同时我们还提供健康咨询及就医协助等服务帮助已患病用户提升诊疗效率和感受，尽早恢复健康状态。 Example Output: This system collects and stores health data, analyzes disease trends, implements a three-tier real-time early warning system, and provides early disease screening tools. It also offers health counseling and medical assistance services.
输入部分	My Input: {APP's core function description}

表 5 大模型判断保活行为良恶

提示词部分	提示词
任务描述	Analyze the following two pieces of information about an Android application and classify its keep-alive behavior as either "Benign" or "Suspicious": 1. **Core Function Description** 2. **Keep-Alive Purpose**
输出要求	Evaluation Criteria: - Classify as "Benign" if the keep-alive behavior directly supports the app's core functionality in a user-beneficial way. - Classify as "Suspicious" if the keep-alive behavior appears unrelated to the stated core functionality, or primarily serves non-essential purposes like aggressive advertising, data collection, or background resource consumption. Response Format: - Only output one of these two classifications followed by a brief rationale (50-100 characters). - Use this format(Categorical judgments should be in parentheses!): [Classification]: Rationale
输出示例	Examples for Reference: 1. Food Delivery App (Core: order tracking) + Keep-alive for order status updates → Benign 2. Calculator App (Core: math operations) + Keep-alive for ad popups → Suspicious Example Output:

	[Suspicious]: Keep-alive for background ops isn't core to privacy protection.
输入部分	My Input: 1. Core Function Description: {APP's core function description} 2. Keep-Alive Purpose: {Keep-Alive Code Purpose}

致 谢

本文不仅体现了本人半年多来针对 Android 保活行为的学习与研究成果,同时也更多地凝聚了老师、同学和朋友们的支持与帮助,在这里,我向他们表示由衷的感谢。

感谢袁巍老师对课题全面细致的指导,袁老师精益求精、严谨认真的工作作风深深感染和激励着我,从研究工作展开、方向指导、任务书拟定、开题报告审查、PPT 制作一直到论文的完成,都离不开袁老师的悉心指导。希望研究生三年能继续在袁老师的指引下,上下求索,潜心问道。

感谢高翠莹学姐、张瀚文学长对我的鼎力支持,课题的确难度很大,但是有学姐学长的指导,一步步完成富有挑战性的工作,是一件非常有成就感的事情,本科毕设结束,但课题还未结束,希望能继续在学姐学长的指导下,继续对这个课题进行深入探究与改进,圆满完成我的学术第一站。

感谢防伪中心二楼袁老师实验室的学长学姐和同学们,很庆幸选择了袁老师实验室,这是一个有爱的大家庭,期待未来和实验室大家的共同努力!

最后,感谢大学四年以来培养栽培我的电信学院和 Dian 团队,感谢多年来父母和长辈对我的默默支持,感谢通信一班汤雅琴同学两年来持续的守望相助、并肩前行,以及,感谢四年来坚定自信、永不言弃的自己。

路漫漫其修远兮,吾将上下而求索。